Object Oriented System Design (KCS-054)		
Course Outcome (CO)		**Bloom's Knowledge Level (KL)**
At the end of course , the student will be able to:		
CO 1	Understand the application development and analyze the insights of object oriented programming to implement application	K_2, K_4
CO 2	Understand, analyze and apply the role of overall modeling concepts (i.e. System, structural)	K_2, K_3
CO 3	Understand, analyze and apply oops concepts (i.e. abstraction, inheritance)	K_2, K_3, K_4
CO 4	Understand the basic concepts of C++ to implement the object oriented concepts	K_2, K_3
CO 5	To understand the object oriented approach to implement real world problem.	K_2, K_3

DETAILED SYLLABUS		**3-0-0**
Unit	**Topic**	**Proposed Lecture**
I	**Introduction:** The meaning of Object Orientation, object identity, Encapsulation, information hiding, polymorphism, generosity, importance of modelling, principles of modelling, object oriented modelling, Introduction to UML, conceptual model of the UML, Architecture.	**08**
II	**Basic Structural Modeling:** Classes, Relationships, common Mechanisms, and diagrams. Class &Object Diagrams: Terms, concepts, modelling techniques for Class & Object Diagrams. **Collaboration Diagrams:** Terms, Concepts, depicting a message, polymorphism in collaboration Diagrams, iterated messages, use of self in messages. Sequence Diagrams: Terms, concepts, depicting asynchronous messages with/without priority, call-back mechanism, broadcast messages. **Basic Behavioural Modeling:** Use cases, Use case Diagrams, Activity Diagrams, State Machine , Process and thread, Event and signals, Time diagram, interaction diagram, Package diagram. **Architectural Modeling:** Component, Deployment, Component diagrams and Deployment diagrams.	**08**
III	Object Oriented Analysis: Object oriented design, Object design, Combining three models, Designing algorithms, design optimization, Implementation of control, Adjustment of inheritance, Object representation, Physical packaging, Documenting design considerations. **Structured analysis and structured design (SA/SD),** Jackson Structured Development (JSD).Mapping object oriented concepts using non-object oriented language, Translating classes into data structures, Passing arguments to methods, Implementing inheritance, associations encapsulation. **Object oriented programming style:** reusability, extensibility, robustness, programming in the large. Procedural v/s OOP, Object oriented language features. Abstraction and Encapsulation.	**08**
IV	**C++ Basics :** Overview, Program structure, namespace, identifiers, variables, constants, enum, operators, typecasting, control structures **C++ Functions :** Simple functions, Call and Return by reference, Inline functions, Macro Vs. Inline functions, Overloading of functions, default arguments, friend functions, virtual functions	**08**
V	**Objects and Classes :** Basics of object and class in C++, Private and public members, static data and function members, constructors and their types, destructors, operator overloading, type conversion. Inheritance : Concept of Inheritance, types of inheritance: single, multiple, multilevel, hierarchical, hybrid, protected members, overriding, virtual base class **Polymorphism :** Pointers in C++, Pointes and Objects, this pointer, virtual and pure virtual functions, Implementing polymorphism	**08**

Text Books

1. James Rumbaugh et. al, "Object Oriented Modeling and Design", Pearson Education
2. Grady Booch, James Rumbaugh, Ivar Jacobson, "The Unified Modeling Language User Guide", Pearson Education
3. Object Oriented Programming With C++, E Balagurusamy, McGraw Hill.
4. C++ Programming, Black Book, Steven Holzner, dreamtech
5. Object Oriented Programming in Turbo C++, Robert Lafore, Galgotia
6. Object Oriented Programming with ANSI and Turbo C++, Ashok Kamthane, Pearson
7. The Compete Reference C++, Herbert Schlitz, McGraw Hill.

Unit II

To model a system at different levels of abstraction by presenting diagrams with different levels of detail.

1. Lower Level of Abstraction gives detailed view
2. Higher Level of Abstraction hides detailed view

So,create a diagram at the right level of abstraction by hiding or revealing the following four categories of things from your model:

1. Building blocks and relationships: Hide those that are not relevant to the intent of your diagram or the needs of your reader.

2. Adornments: Reveal only the adornments of these building blocks and relationships that are essential to understanding your intent.

3. Flow: Expand only those messages or transitions that are essential to understanding your intent.

4. Stereotypes: Used to classify lists of things, such as attributes and operations, reveal only those stereotyped items that are essential to understanding your intent.

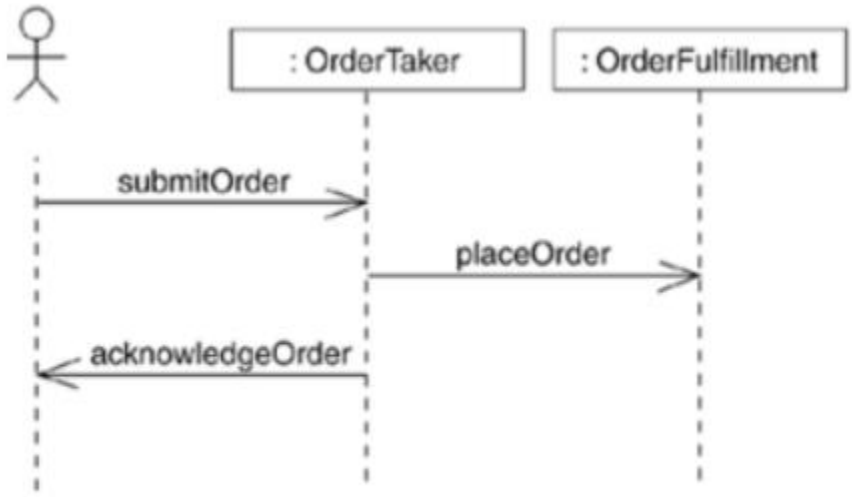
In practice, if you follow the five views of architecture, there are four common situations you'll encounter when modeling a system at different levels of abstraction:

1.Use cases and their realization: Use cases in a use case model will trace to collaborations in a design model.

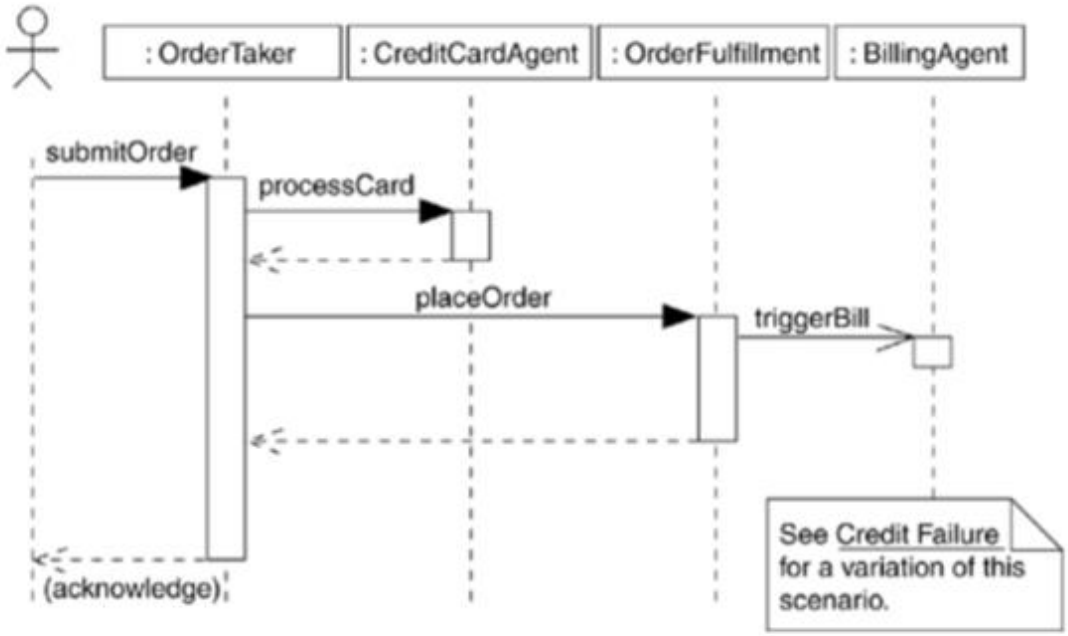
2.Collaborations and their realization: Collaborations will trace to a society of classes that work together to carry out the collaboration.

3.Components and their design: Components in an implementation model will trace to the elements in a design model.

4.Nodes and their components: Nodes in a deployment model will trace to components in an implementation model.



Higher Level of Abstraction



Lower level of Abstraction

Structural Diagrams

The UML's structural diagrams exist to visualize, specify, construct, and document the static aspects of a system. You can think of the static aspects of a system as representing its relatively stable skeleton and scaffolding.The static aspects of a software system encompass the existence and placement of such things as classes, interfaces, collaborations, components, and nodes.

The UML's structural diagrams are roughly organized around the major groups of things you'll find when modeling a system.
1.Class diagram -Classes, interfaces, and collaborations
2.Component diagram -Components
3.Object diagram -Objects
4.Deployment diagram -Nodes

Behavioural Diagrams

The UML's behavioural diagrams are used to visualize, specify, construct, and document the dynamic aspects of a system. You can think of the dynamic aspects of a system as representing its changing parts.The dynamic aspects of a software system encompass things as the flow of messages over time and the physical movement of components across a network.

The UML's behavioural diagrams are roughly organized around the major ways you can model the dynamics of a system.
1.Use case diagram -Organizes the behaviours of the system
2.Sequence diagram- Focuses on the time ordering of messages
3.Collaboration diagram -Focuses on the structural organization of objects that send and receive
messages
4.State diagram-Focuses on the changing state of a system driven by events
5.Activity diagram-Focuses on the flow of control from activity to activity

It is a diagram that shows the configuration of run time processing nodes and the components that live on them. Deployment diagrams is a kind of structure diagram used in modeling the physical aspects of an object-oriented system. They are often be used to model the static deployment view of a system (topology of the hardware).

Purpose of Deployment Diagrams

They show the structure of the run-time system
They capture the hardware that will be used to implement the system and the links between different items of hardware.
They model physical hardware elements and the communication paths between them
They can be used to plan the architecture of a system.
They are also useful for Document the deployment of software components or nodes

Deployment diagram is just a special kind of class diagram, which focuses on a system's nodes. Graphically, a deployment diagram is a collection of vertices and arcs. Deployment diagrams commonly contain:

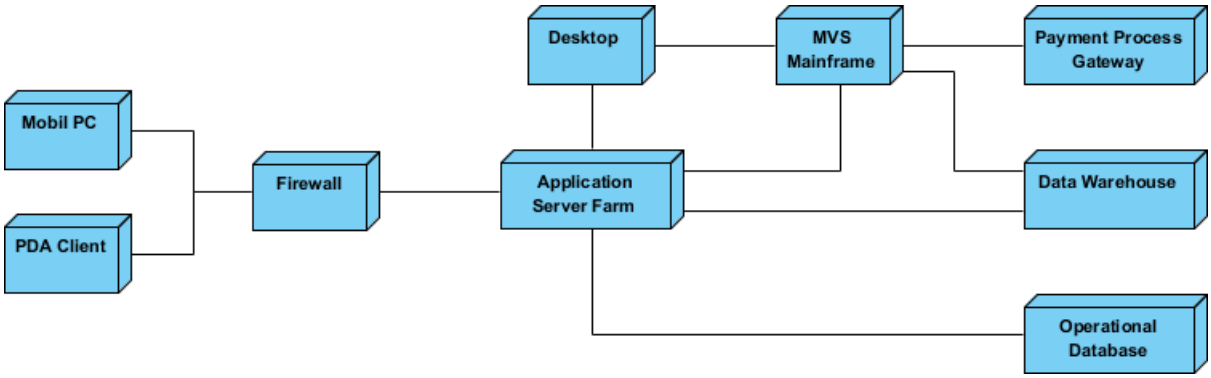
Nodes

- 3-D box represents a node, either software or hardware
- HW node can be signified with <<stereotype>>
- Connections between nodes are represented with a line, with optional <<stereotype>>
- Nodes can reside within a node

Other Notations

- Dependency
- Association relationships.
- May also contain notes and constraints.

Deployment Diagram Example - Corporate Distributed System



Object diagrams model the instances of things contained in class diagrams. An object diagram shows a set of objects and their relationships at a point in time.It shows a set of objects and their relationships at a point in time. Graphically, an object diagram is a collection of vertices and arcs.

Common Properties:

An object diagram is a special kind of diagram and shares the same common properties as all other diagrams that are, a name and graphical contents that are a projection into a model. What distinguishes an object diagram from all other kinds of diagrams is its particular content.

Contents:

1. Objects

2. Links

Like all other diagrams, object diagrams may contain notes and constraints.

Common Uses:

1.To model the static design view or static process view of a system . This view primarily supports the functional requirements of a system that is, the services the system should provide to its end users. Object diagrams let you model static data structures.

2.To model object structures

Modeling object structures involves taking a snapshot of the objects in a system at a given moment in time. An object diagram represents one static frame in the dynamic storyboard represented by an interaction diagram.

3.Use object diagrams to visualize, specify, construct, and document the existence of certain instances in your system, together with their relationships to one another.

It is a diagram that shows a set of classes, interfaces, and collaborations and their relationships. Graphically, a class diagram is a collection of vertices and arcs.

Common Properties:

A class diagram is a special kind of diagram and shares the same common properties as do all other diagrams a name and graphical content that are a projection into a model.

Contents:

Class diagrams commonly contain the following things:

1. Classes
2. Interfaces
3. Dependency, generalization, and association relationships

Like all other diagrams, class diagrams may contain notes and constraints. Class diagrams may also contain packages or subsystems, both of which are used to group elements of your model into larger chunks.

Common Uses:

1.To model the static design view of a system. This view primarily supports the functional requirements of a system the services the system should provide to its end users.

2. To model the vocabulary of a system

To specify these abstractions and their responsibilities.

3. To model simple collaborations

A collaboration is a society of classes, interfaces, and other elements that work together to provide some cooperative behavior that's bigger than the sum of all the elements. For example, when you are modeling the semantics of a transaction in a distributed system, you can't just stare at a single class to understand what's going on.

4. To model a logical database schema

In many domains, you'll want to store persistent information in a relational database or in an object-oriented database. You can model schemas for these databases using class diagrams.

To model a system from different views,

- Decide which views you need to best express the architecture of your system and to expose the technical risks to your project.
- For each of these views, decide which artifacts you need to create to capture the essential details of that view.
- As part of your process planning, decide which of these diagrams you'll want to put under some sort of formal or semi-formal control.
- Allow room for diagrams that are thrown away.

For example, if you are modeling a simple monolithic application that runs on a single machine, you might need only the following handful of diagrams.

Use case view- Use case diagrams
Design view- Class diagrams (for structural modeling)
Interaction view -Interaction diagrams (for behavioral modeling)
Implementation view- Composite structure diagrams
Deployment view- None required

Similarly, if yours is a client/server system, you'll probably want to include component diagrams and deployment diagrams to model the physical details of your system.

Finally, if you are modeling a complex, distributed system, you'll need to employ the full range of the UML's diagrams in order to express the architecture of your system and the technical risks to your project, as in the following.

Use case view

1. Use case diagrams
2. Sequence diagrams

Design view

1. Class diagrams (for structural modeling)
2. Interaction diagrams (for behavioral modeling)
3. State diagrams (for behavioral modeling)
4. Activity diagrams

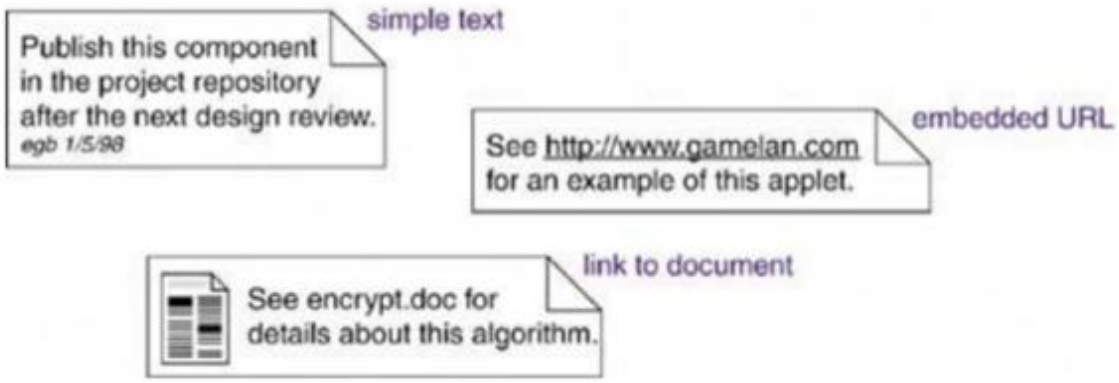
Interaction view

1. Interaction diagrams (for behavioral modeling)
2. Implementation view-
3. Class diagrams
4. Composite structure diagrams

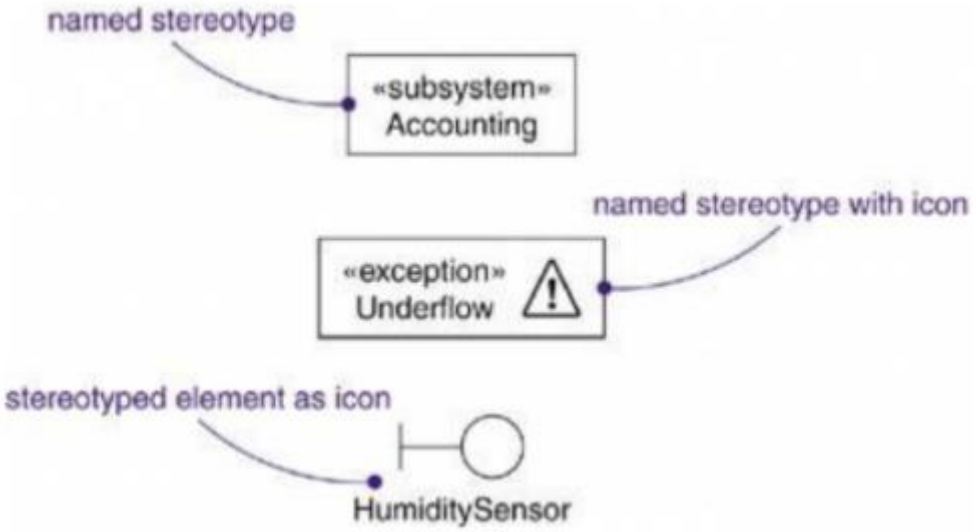
Deployment view

1. Deployment diagrams
2. A **system** is a collection of subsystems organized to accomplish a purpose and described by a set of models, possibly from different viewpoints.
3. A **subsystem** is a grouping of elements, some of which constitute a specification of the behaviour offered by the other contained elements.
4. A **model** is a semantically closed abstraction of a system, meaning that it represents a complete and self-consistent simplification of reality, created in order to better understand the system. In the context of architecture,
5. A **view** is a projection into the organization and structure of a system's model, focused on one aspect of that system.
6. A **diagram** is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships).
7. *Typically, you'll view the static parts of a system using one of the following diagrams.*
8. *1. Class diagram*
9. *2. Component diagram*

10. *3. Composite structure diagram*
11. *4. Object diagram*
12. *5. Deployment diagram*
13. *6. Artifact diagram*
14. *You'll often use five additional diagrams to view the dynamic parts of a system.*
15. *1. Use case diagram*
16. *2. Sequence diagram*
17. *3. Communication diagram*
18. *4. State diagram*
19. *5. Activity diagram*
20. **Note:**A note is a graphical symbol for rendering constraints or comments attached to an element or a collection of elements. Graphically, a note is rendered as a rectangle with a dog-eared corner, together with a textual or graphical comment.A note may contain any combination of text or graphics.

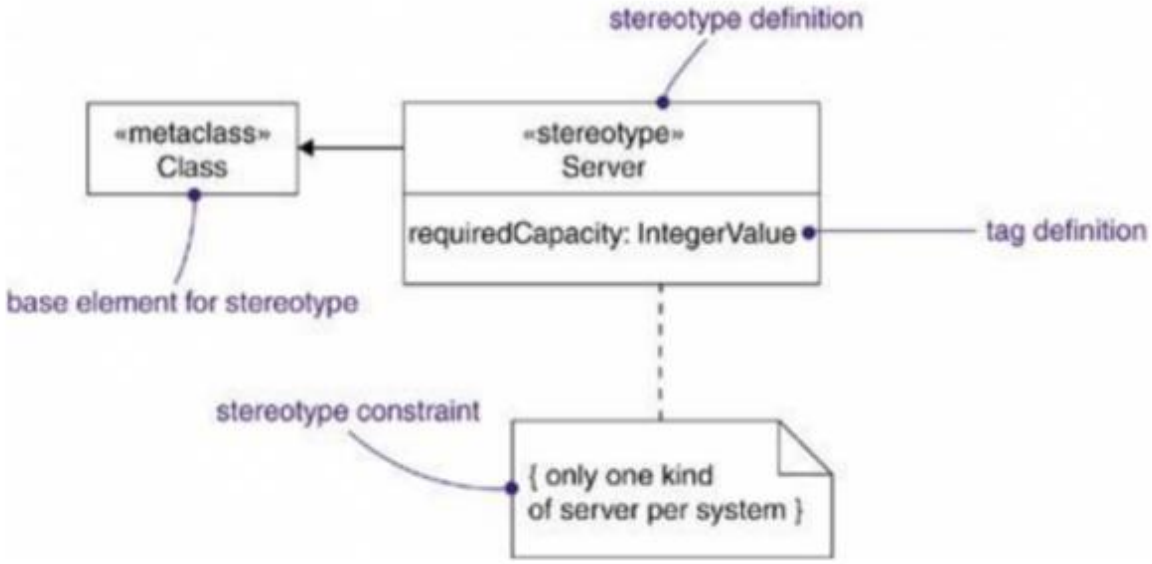


21.
22. **Stereotype:**A stereotype is an extension of the vocabulary of the UML, allowing you to create new kinds of building blocks similar to existing ones but specific to your problem. Graphically, a stereotype is rendered as a name enclosed by guillemets (« »), placed above the name of another element.The UML provides a language for structural things, behavioral things, grouping things, and notational things.



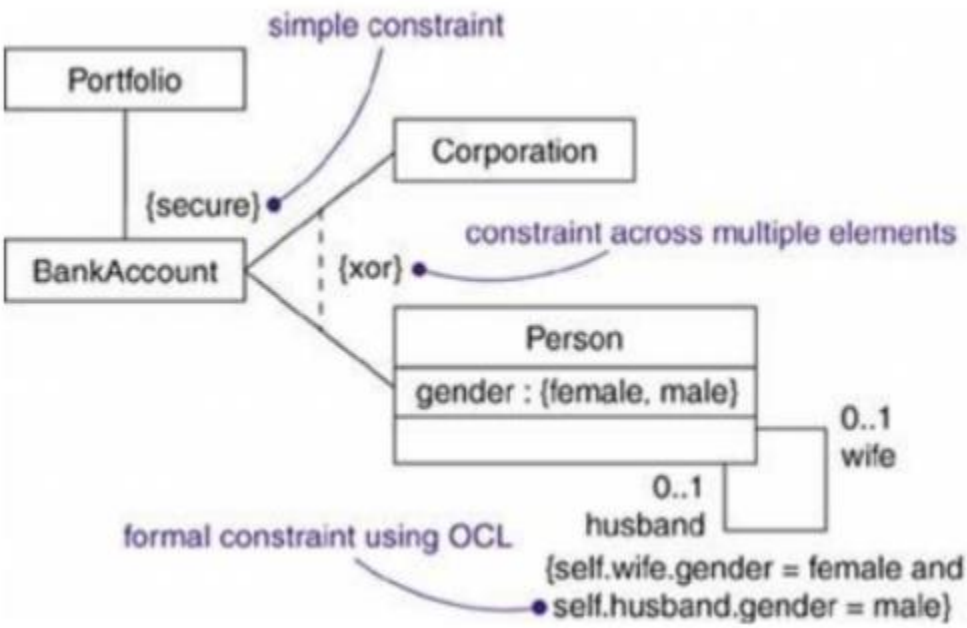
23.

24. **Tagged Value:**A tagged value is a property of a stereotype, allowing you to create new information in an element bearing that stereotype. Graphically, a tagged value is rendered as a string of the form name = value within a note attached to the object.

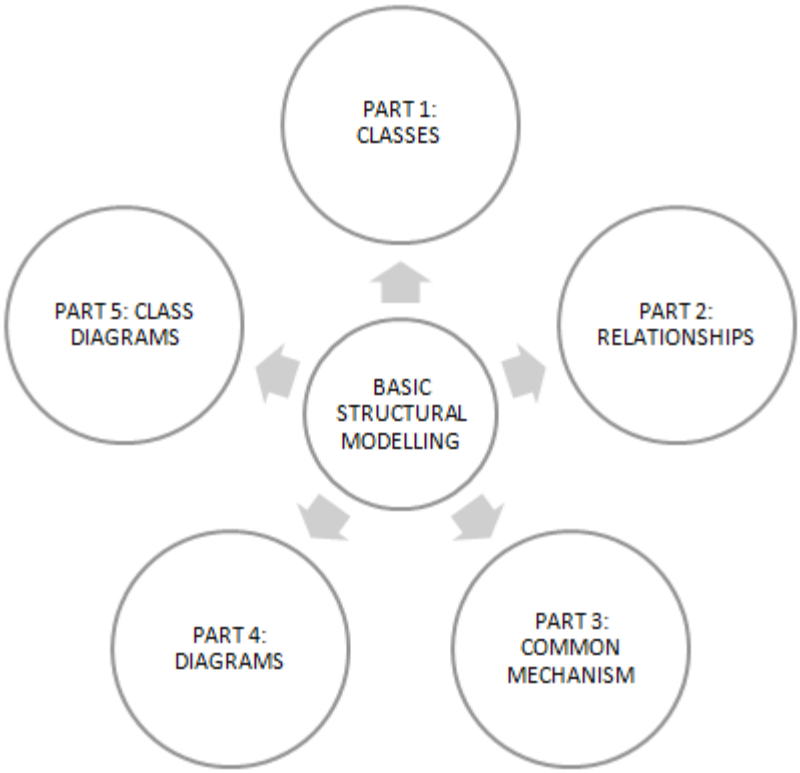


25.

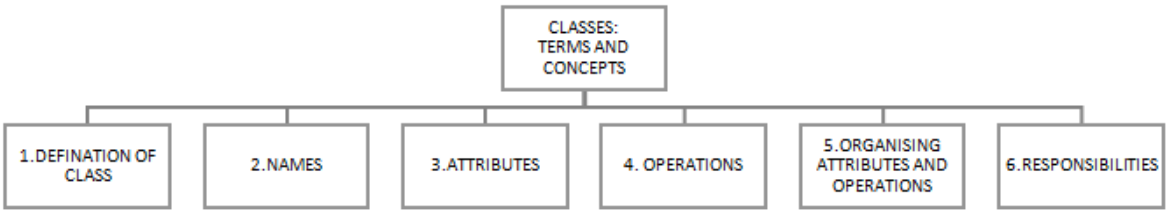
26. **Constraint:**A constraint is a textual specification of the semantics of a UML element, allowing you to add new rules or to modify existing ones. Graphically, a constraint is rendered as a string enclosed by brackets and placed near the associated element or connected to that element or elements by dependency relationships.A constraint specifies conditions that a run-time configuration must satisfy to conform to the model.


simple constraint
Portfolio
Corporation
{secure}
constraint across multiple elements
BankAccount
{xor}
Person
gender : {female, male}
0..1
wife
0..1
husband
formal constraint using OCL
{self.wife.gender = female and
self.husband.gender = male}

27.


PART 1:
CLASSES
PART 2:
RELATIONSHIPS
PART 3:
COMMON
MECHANISM
PART 4:
DIAGRAMS
PART 5: CLASS
DIAGRAMS
BASIC
STRUCTURAL
MODELLING

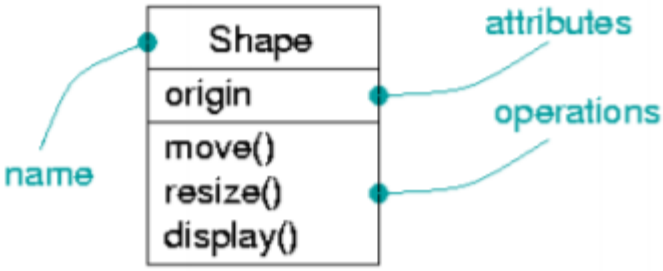
Part 1: **Classes**



1.Definition: A class is a description of a set of objects that share the same attributes, operations, relationships, and semantics.A class can be understood as an abstraction of the things that are part of the application domain vocabulary.

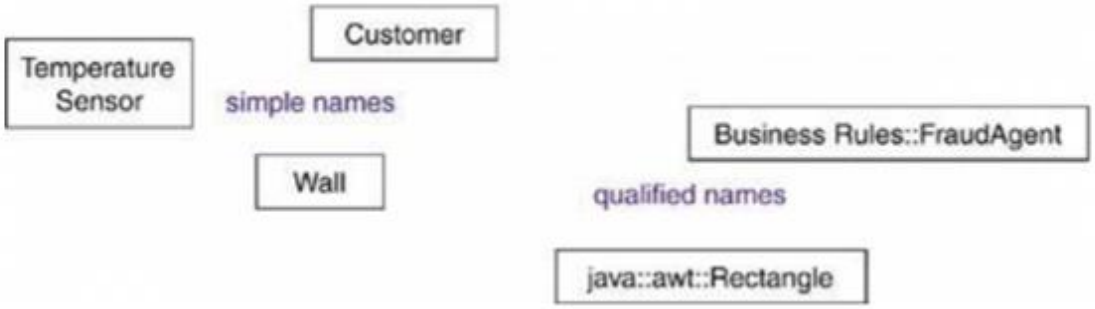
Graphically, a class is rendered as a rectangle.

Can represent software, hardware or conceptual things.



2.Names

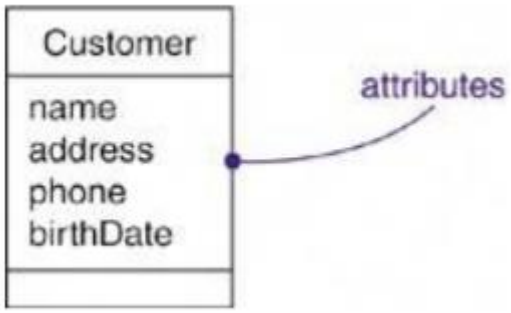
Every class must have a name that distinguishes it from other classes. A name is a textual string. That name alone is known as a simple name; a qualified name is the class name prefixed by the name of the package in which that class lives. A class may be drawn showing only its name.



3.Attributes

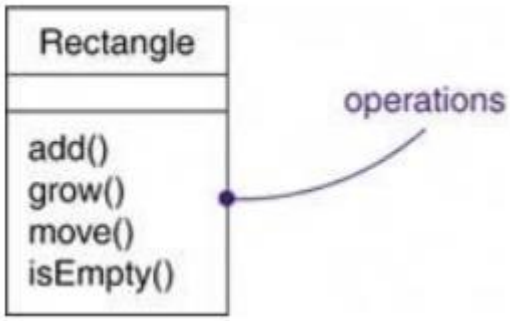
An attribute is a named property of a class that describes a range of values that instances of the property may hold. A class may have any number of

attributes or no attributes at all. An attribute represents some property of the thing you are modeling that is shared by all objects of that class.
For example, every wall has a height, width, and thickness; you might model your customers in such a way that each has a name, address, phone number, and date of birth.

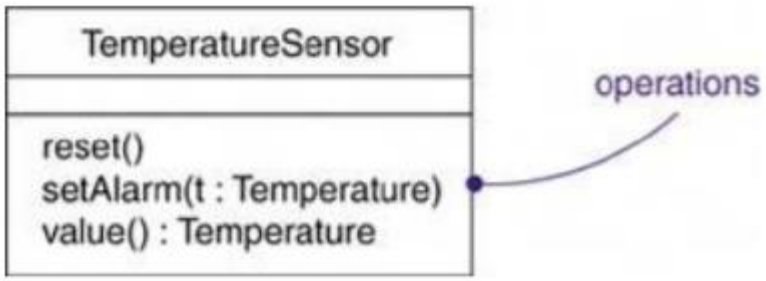


4.Operations

An operation is the implementation of a service that can be requested from any object of the class to affect behavior. In other words, an operation is an abstraction of something you can do to an object that is shared by all objects of that class. A class may have any number of operations or no operations at all. For example, all objects of the class Rectangle can be moved, resized, or queried for their properties.

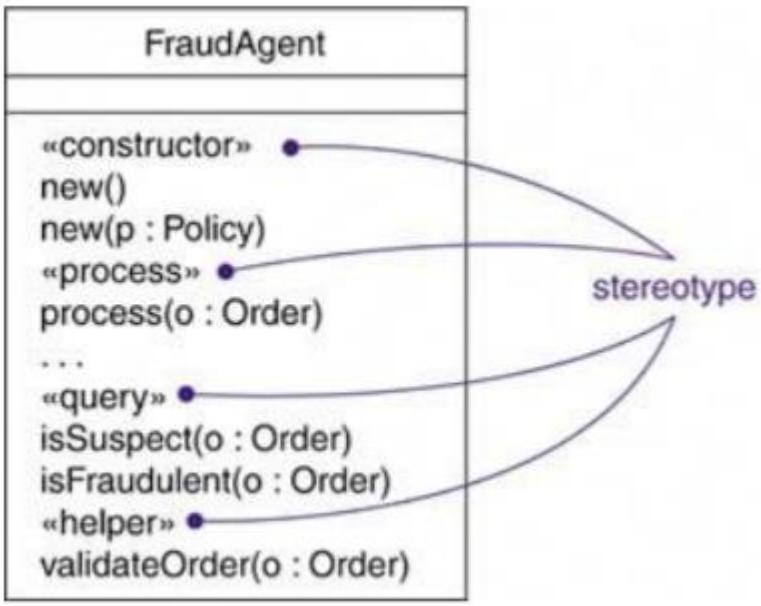


You can specify an operation by stating its signature, which includes the name, type, and default value of all parameters and (in the case of functions) a return type



5. Organizing Attributes and Operations

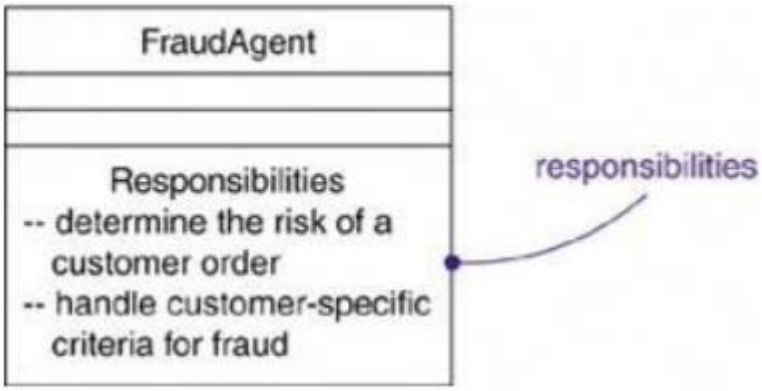
In fact, in most cases, you can't (there are too many of them to put in one figure) and you probably shouldn't (only a subset of these attributes and operations are likely to be relevant to a specific view). For these reasons, you can elide a class, meaning that you can choose to show only some or none of a class's attributes and operations. You can indicate that there are more attributes or properties than shown by ending each list with an ellipsis ("...").



6. Responsibilities

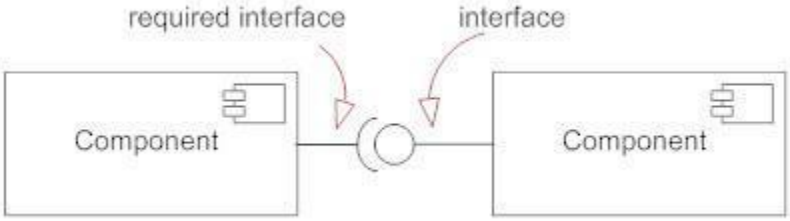
Responsibility is a contract or an obligation of a class. When you create a class, you are making a statement that all objects of that class have the same kind of state and the same kind of behaviour. At a more abstract level, these corresponding attributes and operations are just the features by which the class's responsibilities are carried out.

Example: A Fraud Agent class, as you might find in a credit card application, is responsible for processing orders and determining if they are legitimate, suspect, or fraudulent.

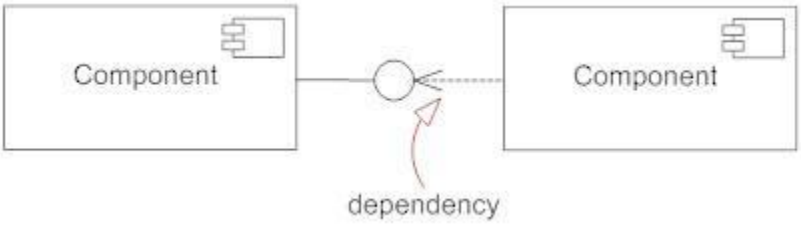


A **component** is a logical unit block of the system, a slightly higher abstraction than classes. It is represented as a rectangle with a smaller rectangle in the upper right corner with tabs or the word written above the name of the component to help distinguish it from a class.

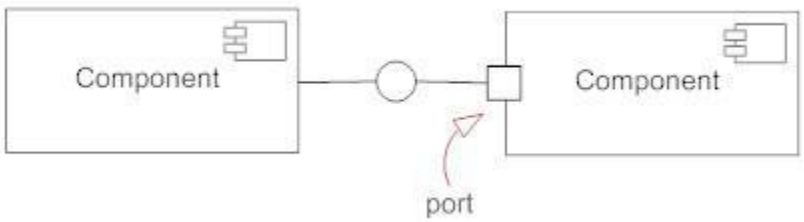
An **interface** (small circle or semi-circle on a stick) describes a group of operations used (required) or created (provided) by components. A full circle represents an interface created or provided by the component. A semi-circle represents a required interface, like a person's input.

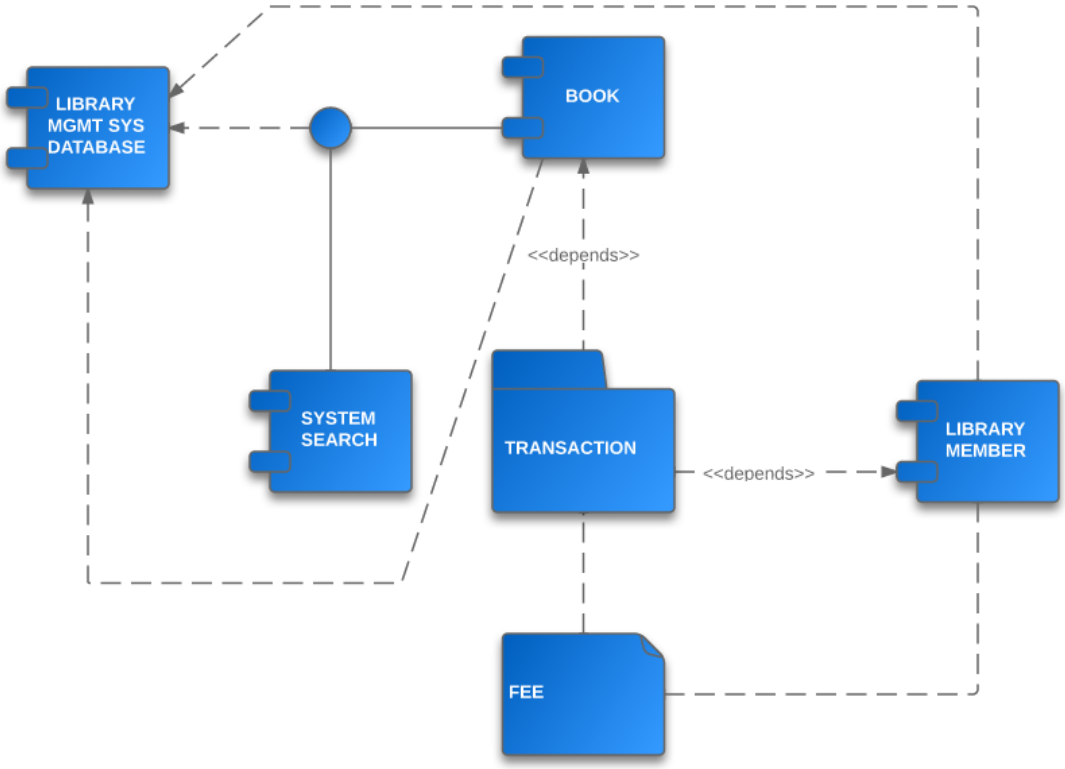


Dependencies among components using dashed arrows.



Ports are represented using a square along the edge of the system or a component. A port is often used to help expose required and provided interfaces of a component.





Notations of Collaboration Diagram

1.Objects

An object is represented by an object symbol showing the name of the object and its class underlined, separated by a colon:

Object_name : class_name

You can use objects in collaboration diagrams in the following ways:
• Each object in the collaboration is named and has its class specified
• Not all classes need to appear
• There may be more than one object of a class
• An object's class can be unspecified. Normally you create a collaboration diagram with objects first and specify their classes later.
• The objects can be unnamed, but you should name them if you want to discriminate different objects of the same class.

Actors

Normally an actor instance occurs in the collaboration diagram, as the invoker of the interaction. If you have several actor instances in the same diagram, try keeping them in the periphery of the diagram.

- Each Actor is named and has a role
- One actor will be the initiator of the use case

Links

Links connect objects and actors and are instances of associations and each link corresponds to an association in the class diagram

Links are defined as follows:

- *A link is a relationship among objects across which messages can be sent. In collaboration diagrams, a link is shown as a solid line between two objects.*
- *An object interacts with, or navigates to, other objects through its links to these objects.*
- *A link can be an instance of an association, or it can be anonymous, meaning that its association is unspecified.*
- *Message flows are attached to links, see Messages.*

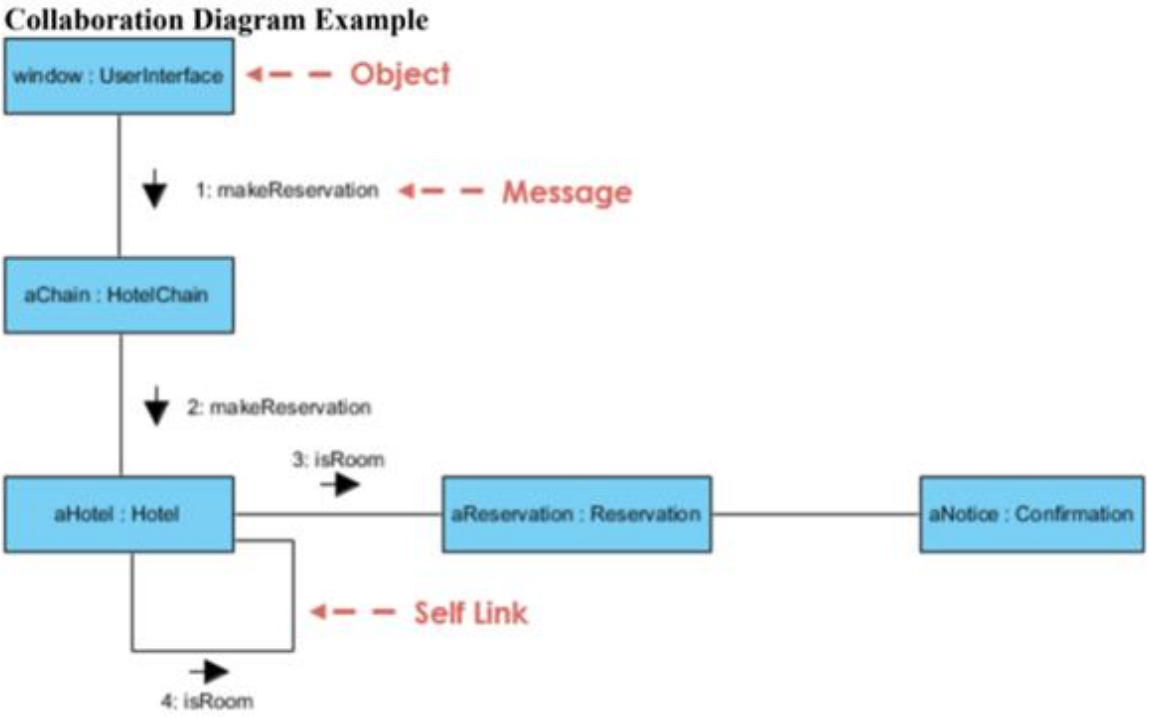
Messages

A message is a communication between objects that conveys information with the expectation that activity will ensue. In collaboration diagrams, a message is shown as a labeled arrow placed near a link.

- *The message is directed from sender to receiver*
- *The receiver must understand the message*
- *The association must be navigable in that direction*

Steps for Creating Collaboration Diagrams

1. Identify behavior whose realization and implementation is specified
2. Identify the structural elements (class roles, objects, subsystems) necessary to carry out the functionality of the collaboration.Decide on the context of interaction: system, subsystem, use case and operation
3. Model structural relationships between those elements to produce a diagram showing the context of the interaction
4. Consider the alternative scenarios that may be required
   - Draw instance level collaboration diagrams, if required.
   - Optionally draw a specification level collaboration diagram to summarize the alternative scenarios in the instance level sequence diagrams



Interaction diagram are used in UML to establish communication between objects.Interaction diagrams mostly focus on message passing and how these messages make up one functionality of a system. Interaction diagrams are designed to display how the objects will realize the particular requirements of a system. The critical component in an interaction diagram is lifeline and messages.

Following are the different types of interaction diagrams defined in UML:
- *Sequence diagram*
- *Collaboration diagram*
- *Timing diagram*

Common Uses:

• To observe the dynamic behaviour of a system.

•Visualizes the communication and sequence of message passing in the system.

• Represents the structural aspects of various objects in the system.

• Represents the ordered sequence of interactions within a system.

• Provides the means of visualizing the real-time data via UML.

• To explain the architecture of an object-oriented or a distributed system.

Contents:

An interaction diagram contains

1. Lifelines
2. Messages
3. Operators
4. State Invariants
5. Constraints

1.Lifeline

A lifeline represents a single participant in an interaction. It describes how an instance of a specific classifier participates in the interaction.

A lifeline represents a role that an instance of the classifier may play in the interaction.

Following are various attributes of a lifeline,

1.1 Name

1. It is used to refer the lifeline within a specific interaction.
2. A name of a lifeline is optional.

1.2 Type

1. It is the name of a classifier of which the lifeline represents an instance.

1.3 Selector

1. It is a Boolean condition which is used to select a particular instance that satisfies the requirement.
2. Selector attribute is also optional.

Messages

A message is a specific type of communication between two lifelines in an interaction. A message involves following activities,

1. A call message which is used to call an operation.
2. A message to create an instance.
3. A message to destroy an instance.
4. For sending a signal.

When a lifeline receives a call message, it acts as a request to invoke an operation that has a similar signature as specified in the message.

Following are the messages used in an interaction diagram:

Message Name	Meaning

Synchronous message	The sender of a message keeps waiting for the receiver to return control from the message execution.
Asynchronous message	The sender does not wait for a return from the receiver; instead,it continues the execution of a next message.
Return message	The receiver of an earlier message returns the focus of control to the sender.
Object creation	The sender creates an instance of a classifier.
Object destruction	The sender destroys the created instance.
Found message	The sender of the message is outside the scope of interaction.
Lost message	The message never reaches the destination, and it is lost in the interaction.

State invariants and constraints

When an instance or a lifeline receives a message, it can cause it to change the state. A state is a condition or a situation during a lifetime of an object at which it satisfies some constraint, performs some operations, and waits for some event.

In interaction diagram, not all messages cause to change the state of an instance. Some messages do not the values of some attribute. It has no side effects on the state of an object.

Operator

An operator specifies an operation on how the operands are going to be executed.

Following are the operators used in an interaction diagram:

Operator	Name	Meaning
Opt	Option	An operand is executed if the condition is true. e.g., If else
Alt	Alternative	The operand, whose condition is true, is executed. e.g., switch

Loop	Loop	It is used to loop an instruction for a specified period.
Break	Break	It breaks the loop if a condition is true or false, and the next instruction is executed.
Ref	Reference	It is used to refer to another interaction.
Par	Parallel	All operands are executed in parallel.

Kind of structural diagram, shows the arrangement and organization of model elements in a middle to large scale project.It can show both structure and dependencies between subsystems or modules, showing different views of a system, for example, as multi-layered application.

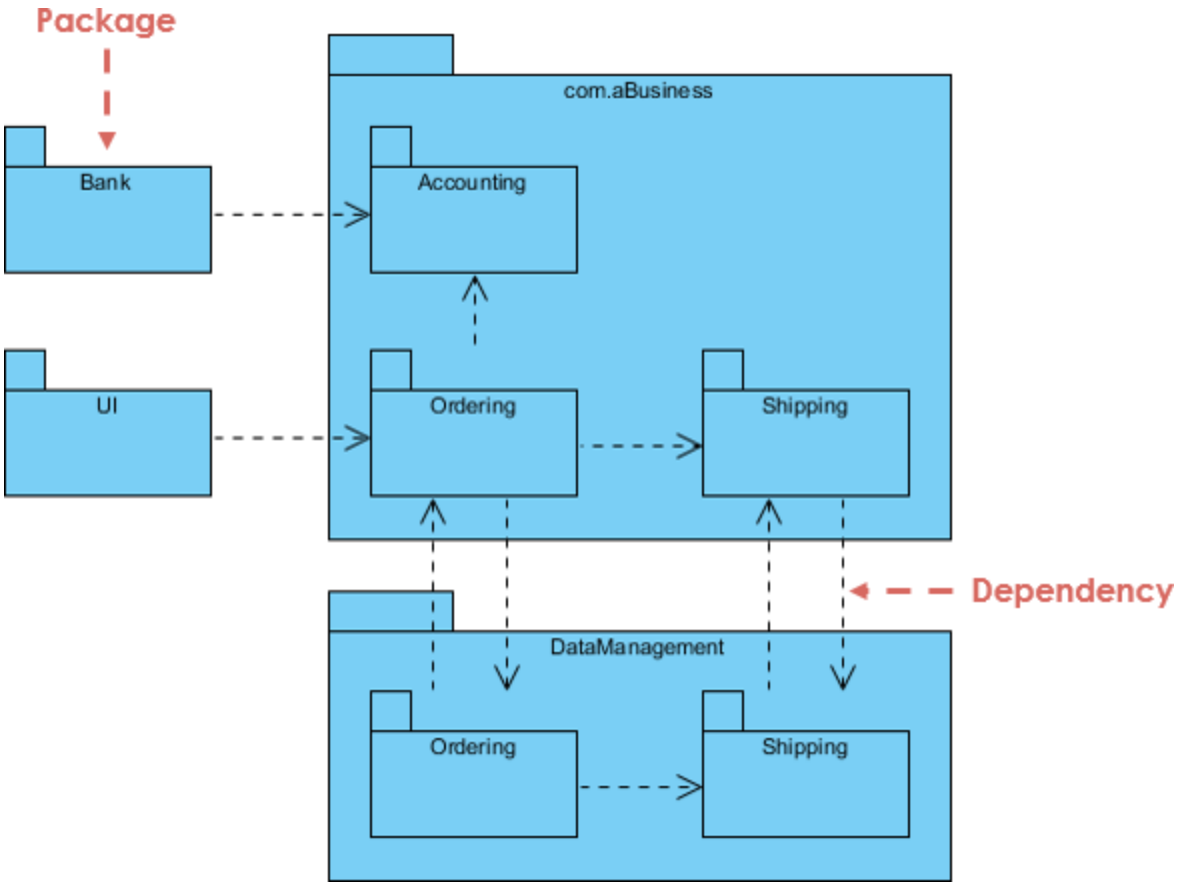
Purpose

Package diagrams are used to structure high level system elements. Packages are used for organizing large system which contains diagrams, documents and other key deliverables.

- Package Diagram can be used to simplify complex class diagrams, it can group classes into packages.
- A package is a collection of logically related UML elements.
- Packages are depicted as file folders and can be used on any of the UML diagrams.
- Used to simplify complex class diagrams, you can group classes into packages. A package is a collection of logically related UML elements.

The diagram below is a business model in which the classes are grouped into packages:

- Packages appear as rectangles with small tabs at the top.
- The package name is on the tab or inside the rectangle.
- The dotted arrows are dependencies.
- One package depends on another if changes in the other could possibly force changes in the first.



Package diagram follows hierarchical structure of nested packages. Atomic modules for nested package are usually class diagrams.

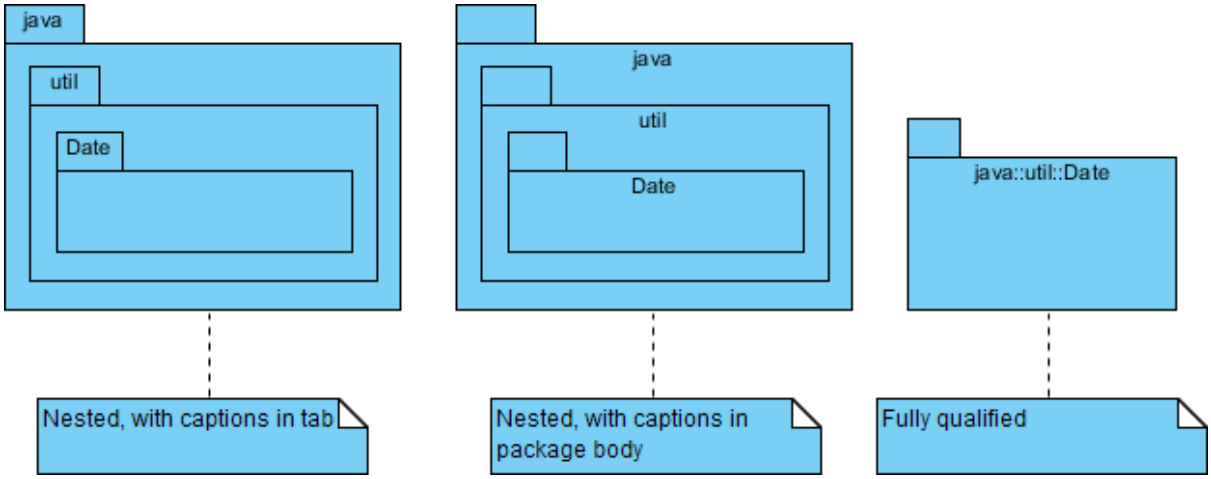
Constraints:

- Package name should not be the same for a system, however classes inside different packages could have the same name.
- Packages can include whole diagrams, name of components alone or no components at all.
- Fully qualified name of a package has the following syntax.

```
Name owing the package : : Name of the package
```

```
java : : util : : Date
```

Packages can be represented by the notations with some examples shown below:

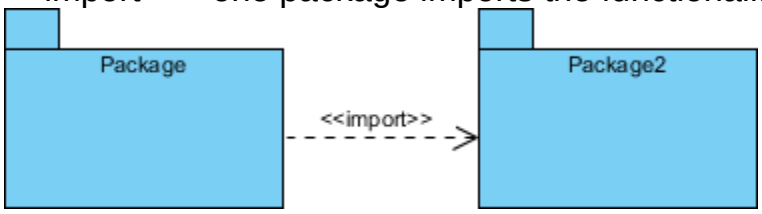


Notation

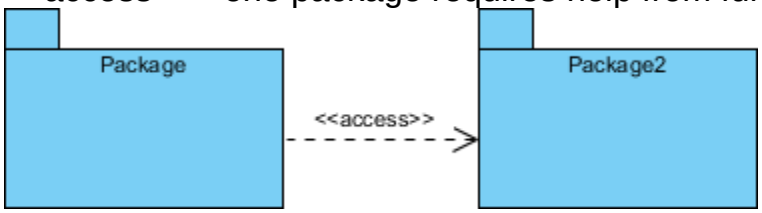
There are two sub-types involved in dependency. They are <<import>> & <<access>>. Though there are two stereotypes users can use their own stereotype to represent the type of dependency between two packages.

Package Diagram Example - Import

<<import>> - one package imports the functionality of other package



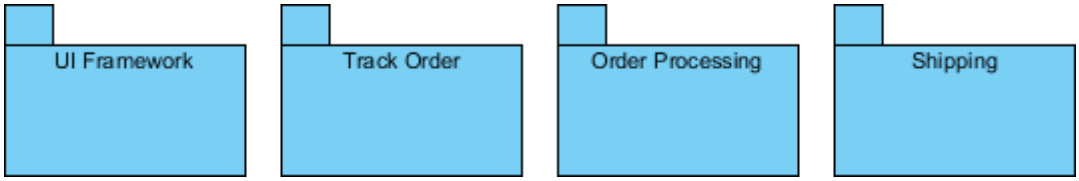
<<access>> - one package requires help from functions of other package.



Based on the project Description we should first identify the packages in the system and then related them together according to the relationship:

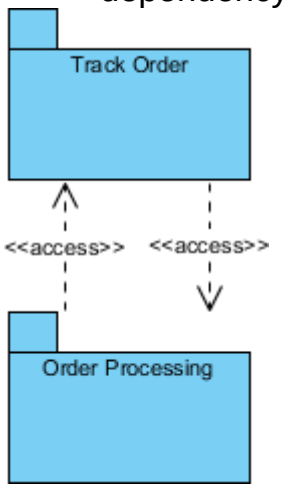
Identify the packages of the system

- There is a track order module, it has to talk with other module to know about the order details, let us call it "Order Details".
- Next after fetching Order Details it has to know about shipping details, let us call that as "Shipping".

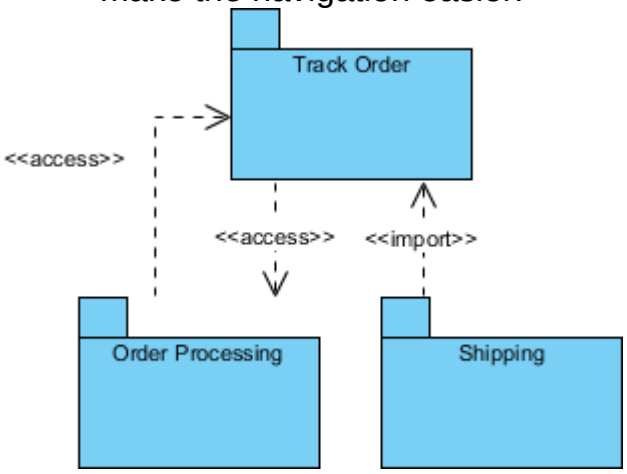


Identify the dependencies in the System

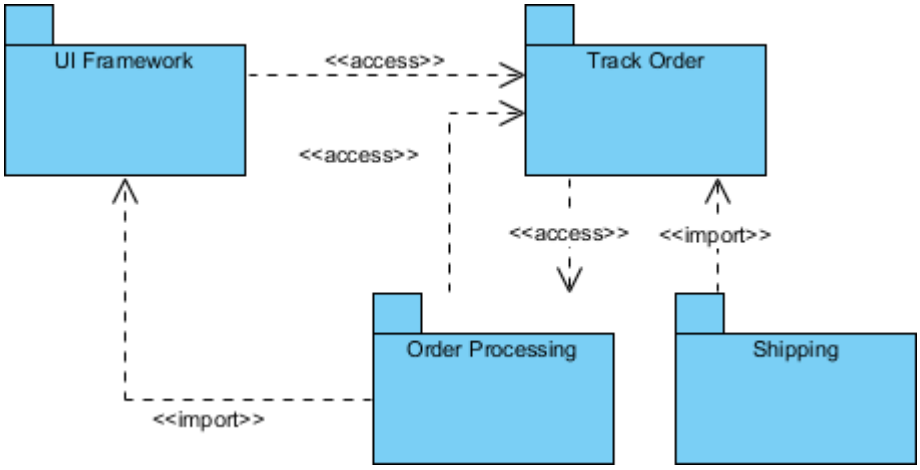
- Track order should get order details from "Order Details" and "Order Details" has to know the tracking info given by the customer. Two modules are accessing each other which suffices <<access>> dual dependency



- To know shipping information, "Shipping" can import "Track Order" to make the navigation easier.



- Finally, Track Order dependency to UI Framework is also mapped which completes our Package Diagram for Order Processing subsystem.

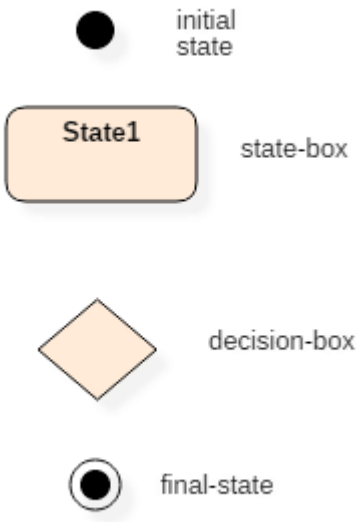


State Machine Diagrams (State diagrams, state machines or state charts) show the different states of an entity. State machine diagrams can also show how an entity responds to various events by changing from one state to another. State machine diagram is a UML diagram used to model the dynamic nature of a system.

Used to describe state-dependent behavior for an object. An object responds differently to the same event depending on what state it is in. State machine diagrams are usually applied to objects but can be applied to any element that has behavior to other entities such as: actors, use cases, methods, subsystems systems and etc. and they are typically used in conjunction with interaction diagrams (usually sequence diagrams).

Notation and Symbol for State Machine

Following are the various notations that are used throughout the state chart diagram. All these notations, when combined, make up a single diagram.



UML state diagram notations

Initial state

The initial state symbol is used to indicate the beginning of a state machine diagram.

Final state

This symbol is used to indicate the end of a state machine diagram.

Decision box

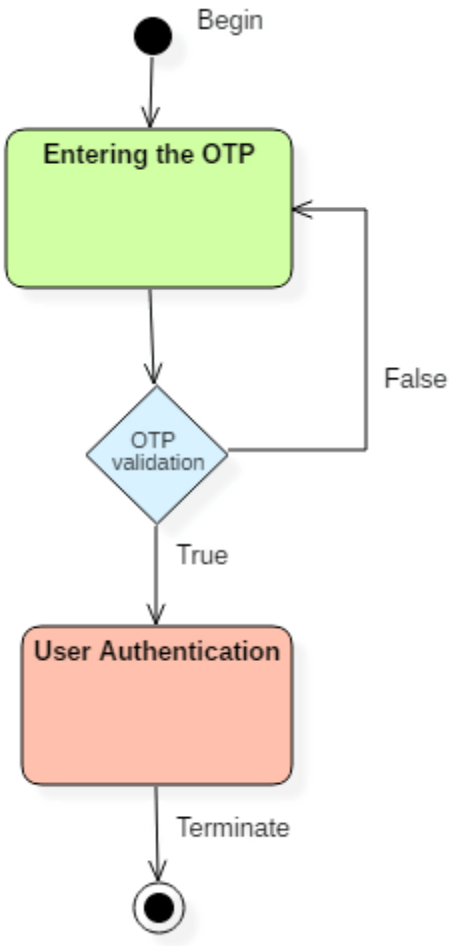
It contains a condition. Depending upon the result of an evaluated guard condition, a new path is taken for program execution.

Transition

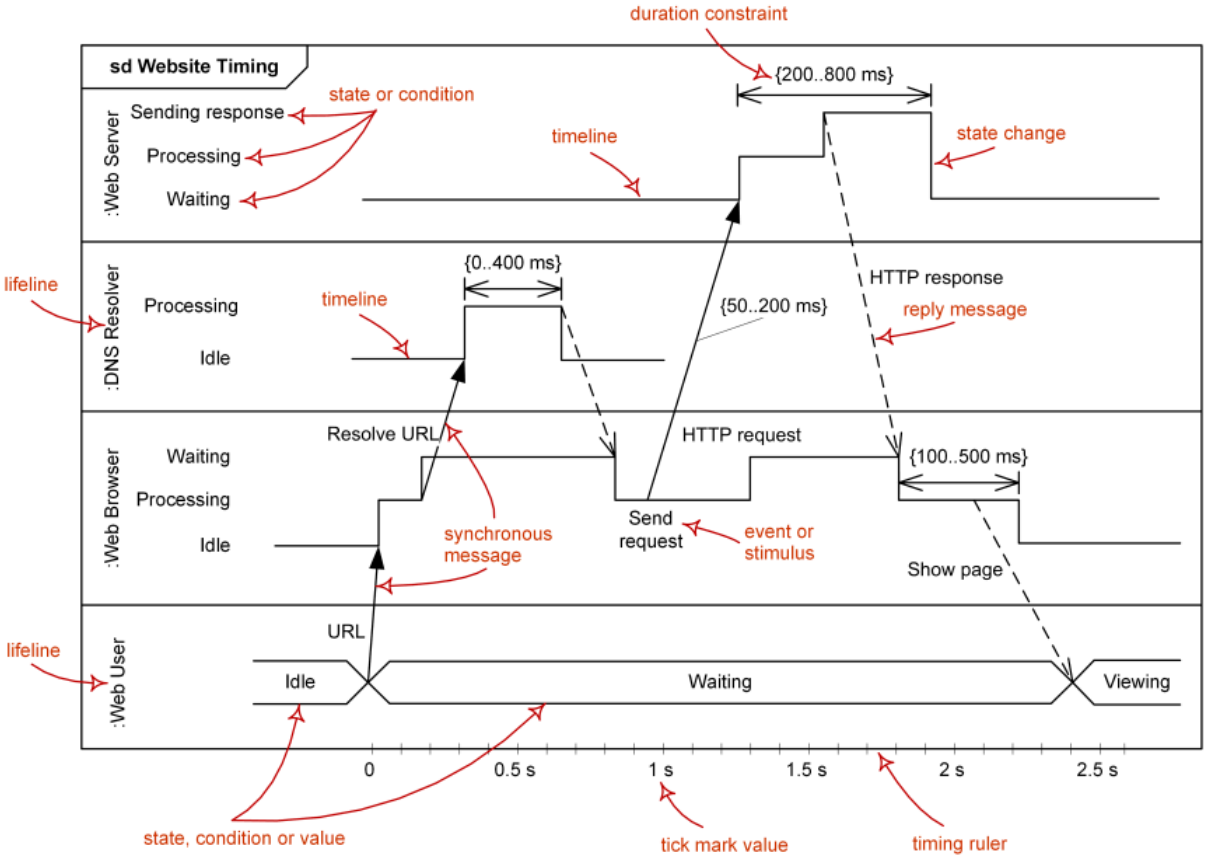
A transition is a change in one state into another state which has occurred because of some event. A transition causes a change in the state of an object.

State box

It is a specific moment in the lifespan of an object. It is defined using some condition or a statement within the classifier body. It is used to represent any static as well as dynamic situations.
It is denoted using a rectangle with round corners. The name of a state is written inside the rounded rectangle.



Timing diagrams focus on conditions changing within and among lifelines along a linear time axis. Timing diagrams describe behavior of both individual classifiers and interactions of classifiers, focusing attention on time of events causing changes in the modeled conditions of the lifelines.

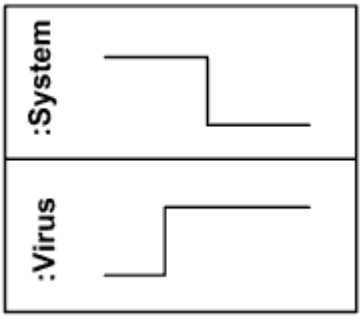


Major elements of timing UML diagram - lifeline, timeline, state or condition, message, duration constraint, timing ruler.

Lifeline

Lifeline is a named element which represents an individual participant in the interaction. While parts and structural features may have multiplicity greater than 1, lifelines represent only one interacting entity. See lifeline from sequence diagrams for details.

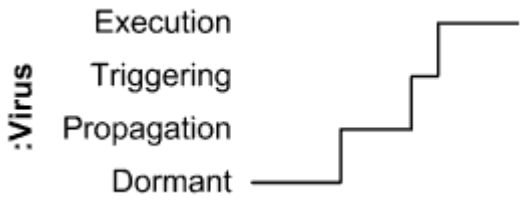
Lifeline on the timing diagrams is represented by the name of classifier or the instance it represents. It could be placed inside a diagram frame or a "swimlane".



Lifelines representing instances of System and Virus

State or Condition Timeline

Timing diagram could show states of the participating classifier or attribute, or some testable conditions, such as a discrete or enumerable value of an attribute.

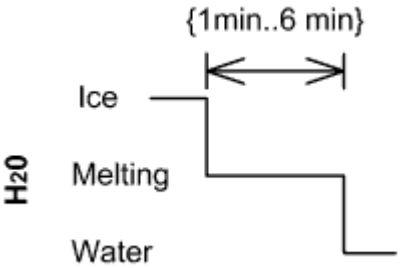


Timeline shows Virus changing its state between Dormant, Propagation, Triggering and Execution state,UML also allows the state/condition dimension be continuous. It could be used in scenarios where entities undergo continuous state changes, such as temperature or density.

Duration Constraint

It is an interval constraint that refers to a duration interval. The duration interval is the duration used to determine whether the constraint is satisfied. The semantics of a duration constraint is inherited from constraints. If constraints are violated, traces become negative which means that system is considered as failed.

Duration constraint is shown as some graphical association between a duration interval and the constructs that it constrains.



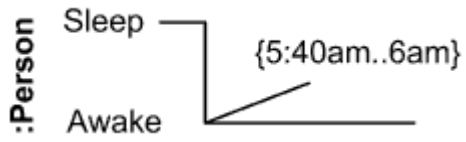
Ice should melt into water in 1 to 6 minutes

Time Constraint

It is an interval constraint that refers to a time interval.

All traces where the constraints are violated are negative traces, i.e., if they occur, the system is considered as failed.

Time constraint is shown as a graphical association between a time interval and the construct that it constrains. Typically this graphical association is a small line, e.g., between an occurrence specification and a time interval.

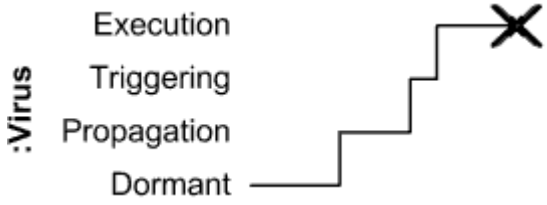


Person should wake up between 5:40 am and 6 am

Destruction Occurrence

It is a message occurrence which represents the destruction of the instance described by the lifeline. It may result in the subsequent destruction of other objects that this object owns by composition.

The destruction event is depicted by a cross in the form of an X at the end of a timeline.



Virus lifeline is terminated

Use cases specify the expected behavior (what), and not the exact method of making it happen (how). A key concept of use case modeling is that it helps us design a system from the end user's perspective. It is an effective technique for communicating system behavior in the user's terms by specifying all externally visible system behavior.

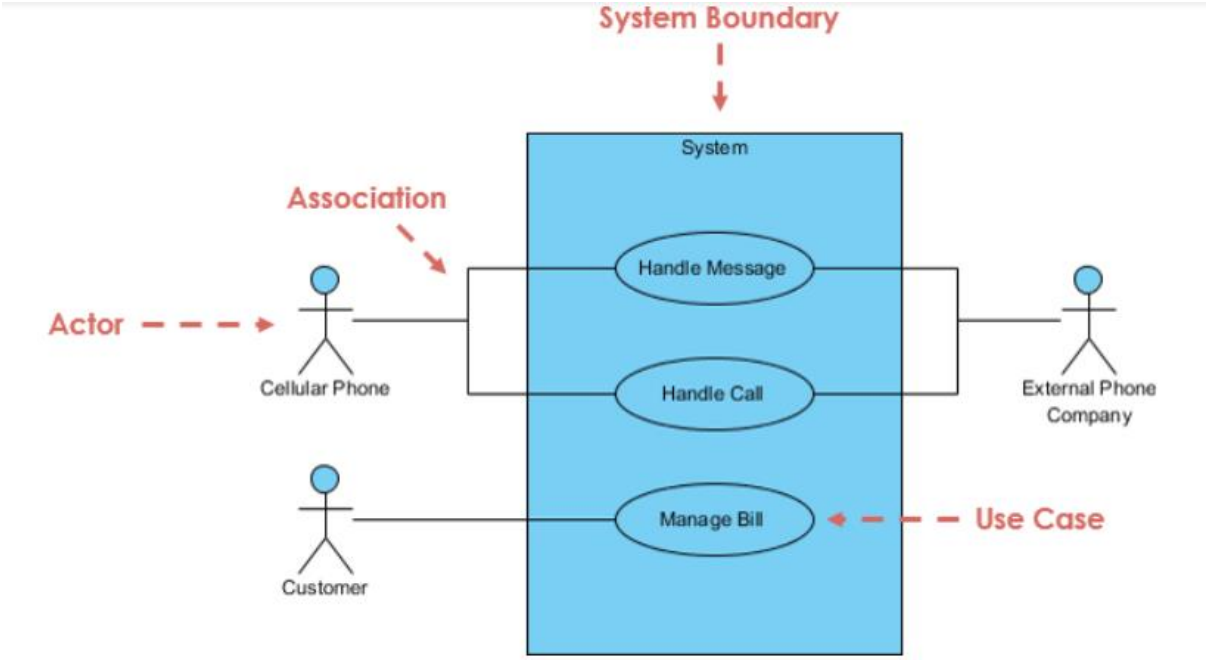
A use case diagram is usually simple. It does not show the detail of the use cases:

- It only summarizes some of the relationships between use cases, actors, and systems.
- It does not show the order in which steps are performed to achieve the goals of each use case.

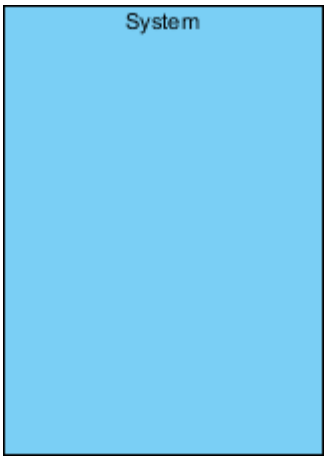
Purpose of Use Case Diagram

Use case diagrams are typically developed in the early stage of development and people often apply use case modeling for the following purposes:

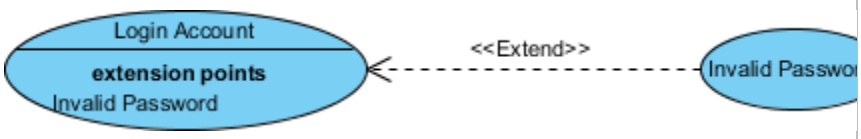
- *Specify the context of a system*
- *Capture the requirements of a system*
- *Validate a systems architecture*
- *Drive implementation and generate test cases*
- *Developed by analysts together with domain experts*

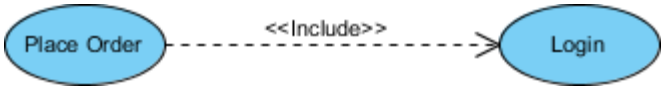


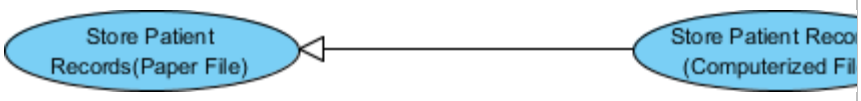
Notation Description	Visual representation
Actor • Someone who interacts with the use case (system function). • Named by a noun. • Actor plays a role in the business • Similar to the concept of user, but a user can play different roles • For example: A prof. can be instructor and also researcher plays 2 roles with two systems • Actor triggers use case(s). • Actor has a responsibility toward the system (inputs), and Actor has expectations from the system (outputs).	

Use Case • System function (process - automated or manual) • Named by verb + Noun (or Noun Phrase). • i.e. Do something • Each Actor must be linked to a use case, while some use cases may not be linked to actors.	
Communication Link • The participation of an actor in a use case is shown by connecting an actor to a use case by a solid link. • Actors may be connected to use cases by associations, indicating that the actor and the use case communicate with one another using messages.	
Boundary of system • The system boundary is potentially the entire system as defined in the requirements document. • For large and complex systems, each module may be the system boundary. • For example, for an ERP system for an organization, each of the modules such as personnel, payroll, accounting, etc. • can form a system boundary for use cases specific to each of these business functions.	

• The entire system can span all of these modules depicting the overall system boundary	

Use case Relationship	Visual Representation
Extends • Indicates that an **"Invalid Password"** use case may include (subject to specified in the extension) the behavior specified by base use case **"Login Account"**. • Depict with a directed arrow having a dotted line. The tip of the arrowhead points to the base use case and the child use case is connected at the base of the arrow. • The stereotype "<<extends>>" identifies as an extend relationship	

Include • When a use case is depicted as using the functionality of another use case, the relationship between the use cases is named as include or uses relationship. • A use case includes the functionality described in another use case as a part of its business process flow. • A uses relationship from base use case to child use case indicates that an instance of the base use case will include the behavior as specified in the child use case. • An include relationship is depicted with a directed arrow having a dotted line.	

The tip of arrowhead points to the child use case and the parent use case connected at the base of the arrow. • The stereotype "<<include>>" identifies the relationship as an include relationship.	
Generalization • A generalization relationship is a parent-child relationship between use cases. • The child use case is an enhancement of the parent use case. • Generalization is shown as a directed arrow with a triangle arrowhead. • The child use case is connected at the base of the arrow.	

The tip of the arrow is connected to the parent use case.	

A sample use case diagram representing the order management system. Hence, if we look into the diagram then we will find three use cases **(Order, SpecialOrder, and NormalOrder)** and one actor which is the customer. The SpecialOrder and NormalOrder use cases are extended from Order use case. Hence, they have extended relationships. Another important point is to identify the system boundary, which is shown in the picture. The actor Customer lies outside the system as it is an external user of the system.

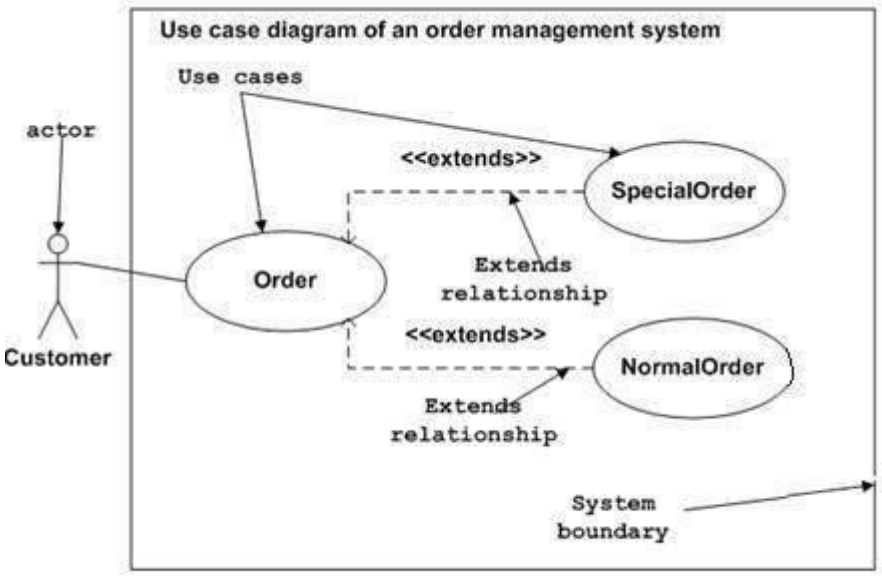


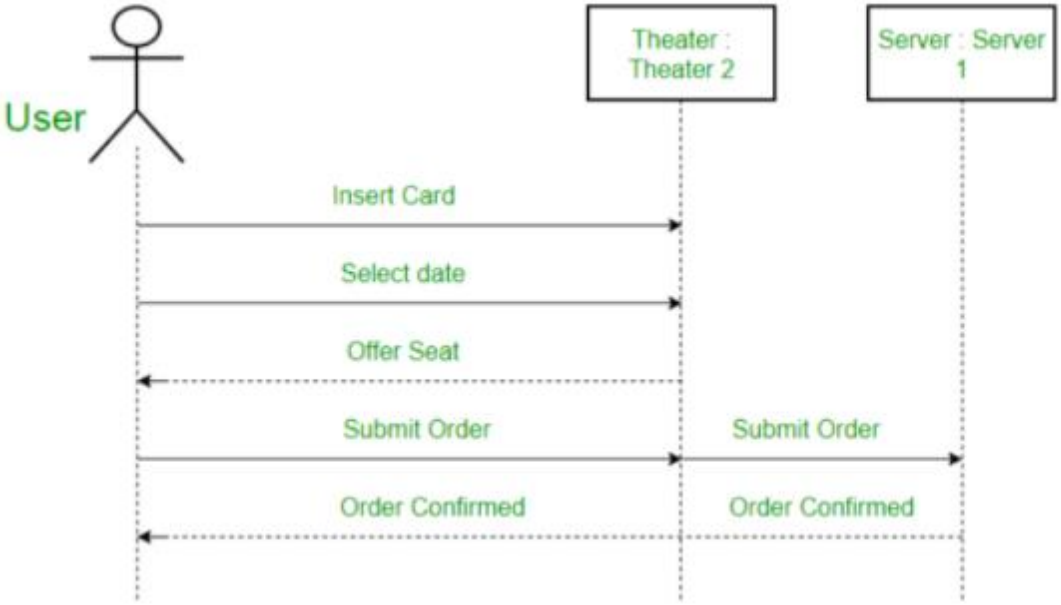
Figure: Sample Use Case diagram

A sequence diagram simply depicts interaction between objects in a sequential order i.e. the order in which these interactions take place. We can also use the terms event diagrams or event scenarios to refer to a sequence diagram. Sequence diagrams describe how and in what order the objects in a system function. These diagrams are widely used by businessmen and software developers to document and understand requirements for new and existing systems.

Notations:

1.Actors – An actor in a UML diagram represents a type of role where it interacts with the system and its objects.

For example – Here the user in seat reservation system is shown as an actor where it exists outside the system and is not a part of the system.
Figure – an actor interacting with a seat reservation system



2.Lifelines: A lifeline is a named element which depicts an individual participant in a sequence diagram. So basically each instance in a sequence diagram is represented by a lifeline.The standard in UML for naming a lifeline follows the following format – Instance Name : Class Name

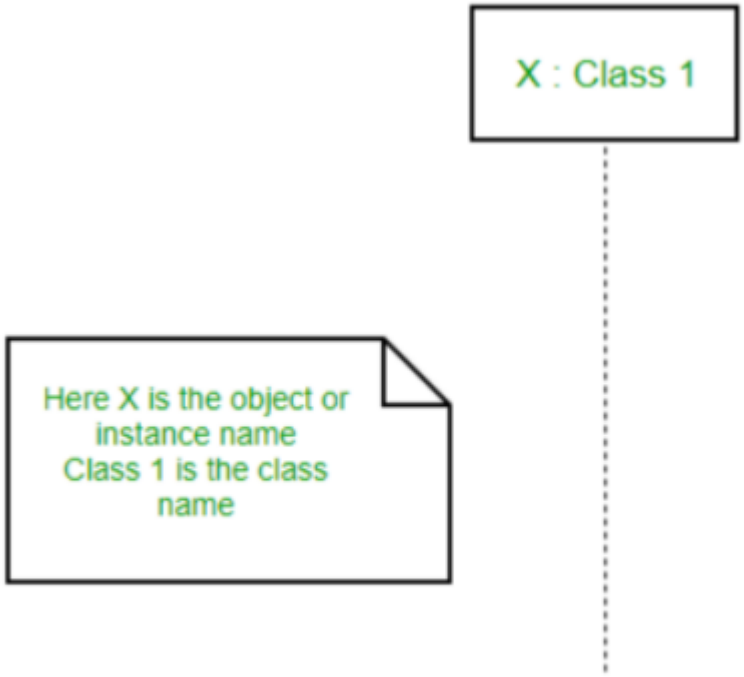


Figure – lifeline

We display a lifeline in a rectangle called head with its name and type. **Difference between a lifeline and an actor:**A lifeline always portrays an object internal to the system whereas actors are used to depicting objects external to the system.

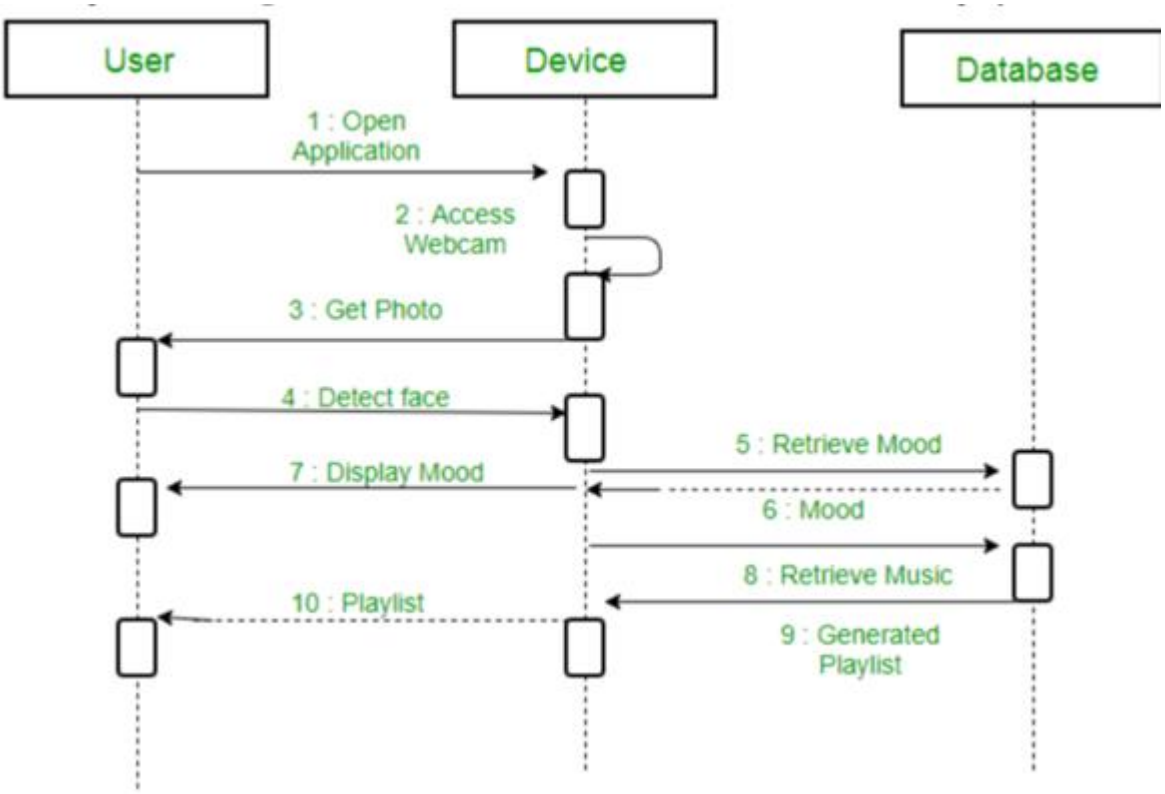


Figure – a sequence diagram for an emotion based music player

The above sequence diagram depicts the sequence diagram for an emotion based music player

1. Firstly the application is opened by the user.
2. The device then gets access to the web cam.
3. The webcam captures the image of the user.
4. The device uses algorithms to detect the face and predict the mood.
5. It then requests database for dictionary of possible moods.
6. The mood is retrieved from the database.
7. The mood is displayed to the user.
8. The music is requested from the database.
9. The playlist is generated and finally shown to the user.

Common Uses:

• Used to model and visualise the logic behind a sophisticated function, operation or procedure.

• They are also used to show details of UML use case diagrams.

• Used to understand the detailed functionality of current or future systems.

• Visualise how messages and tasks move between objects or components in a system.

Unlike a sequence diagram, a collaboration diagram shows the relationships among the objects. Sequence diagrams and collaboration diagrams express similar information, but show it in different ways.

Difference between a lifeline and an actor – A lifeline always portrays an object internal to the system whereas actors are used to depicting objects external to the system.

Messages: Communication between objects is depicted using messages. The messages appear in sequential order on the lifeline. We represent messages using arrows. Lifelines and messages form the core of a sequence diagram. Messages can be broadly classified into the following categories :

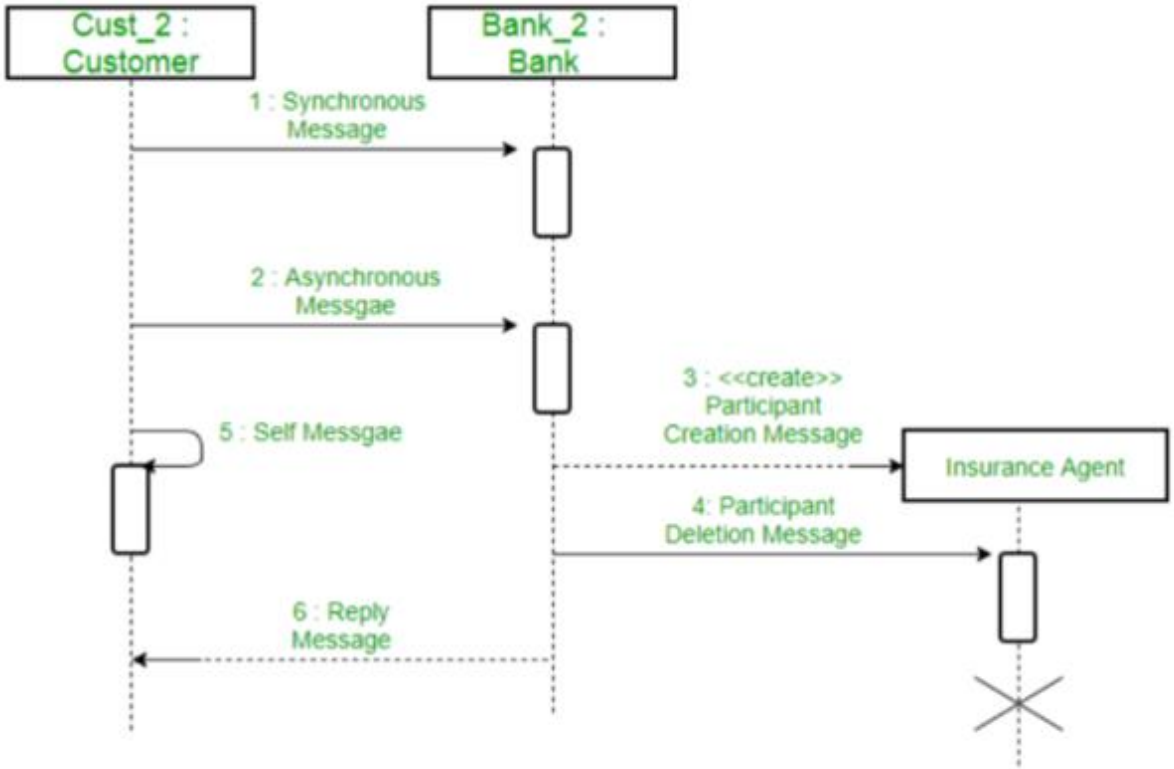
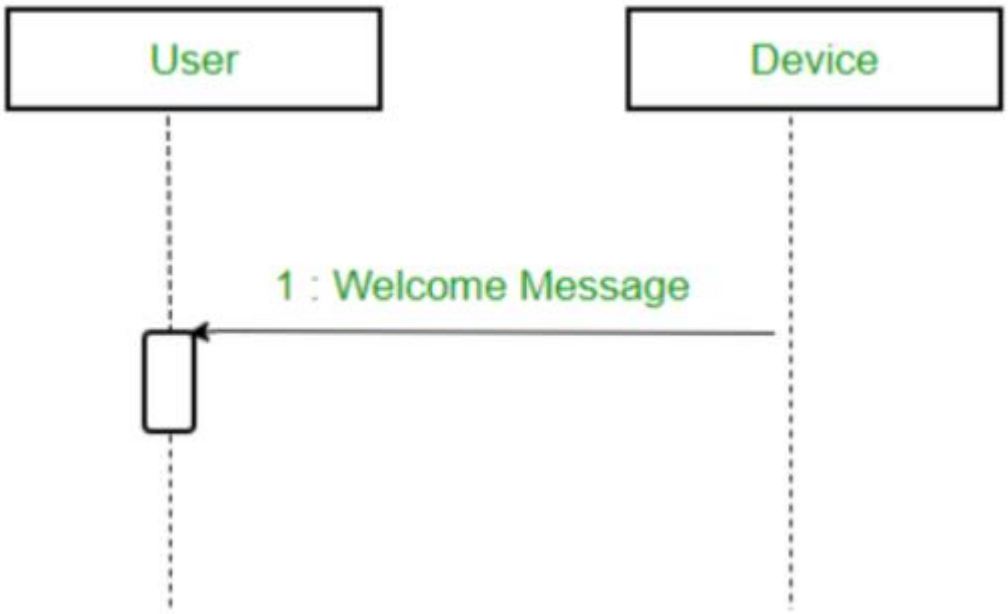


Figure – a sequence diagram with different types of messages

Synchronous messages:A synchronous message waits for a reply before the interaction can move forward. The sender waits until the receiver has completed the processing of the message.

Asynchronous Messages – An asynchronous message does not wait for a reply from the receiver. The interaction moves forward irrespective of the receiver processing the previous message or not. We use a lined arrow head to represent an asynchronous message.



Create message: We use a Create message to instantiate a new object in the sequence diagram. There are situations when a particular message call requires the creation of an object. It is represented with a dotted arrow and create word labelled on it to specify that it is the create Message symbol.

Delete Message: We use a Delete Message to delete an object. When an object is deallocated memory or is destroyed within the system we use the Delete Message symbol. It destroys the occurrence of the object in the system. It is represented by an arrow terminating with a x.

Self Message: Certain scenarios might arise where the object needs to send a message to itself. Such messages are called Self Messages and are represented with a U shaped arrow.

Reply Message: Used to show the message being sent from the receiver to the sender. We represent a return/reply message using an open arrowhead with a dotted line. The interaction moves forward only when a reply message is sent by the receiver.

Found Message:A Found message is used to represent a scenario where an unknown source sends the message. It is represented using an arrow directed towards a lifeline from an endpoint.

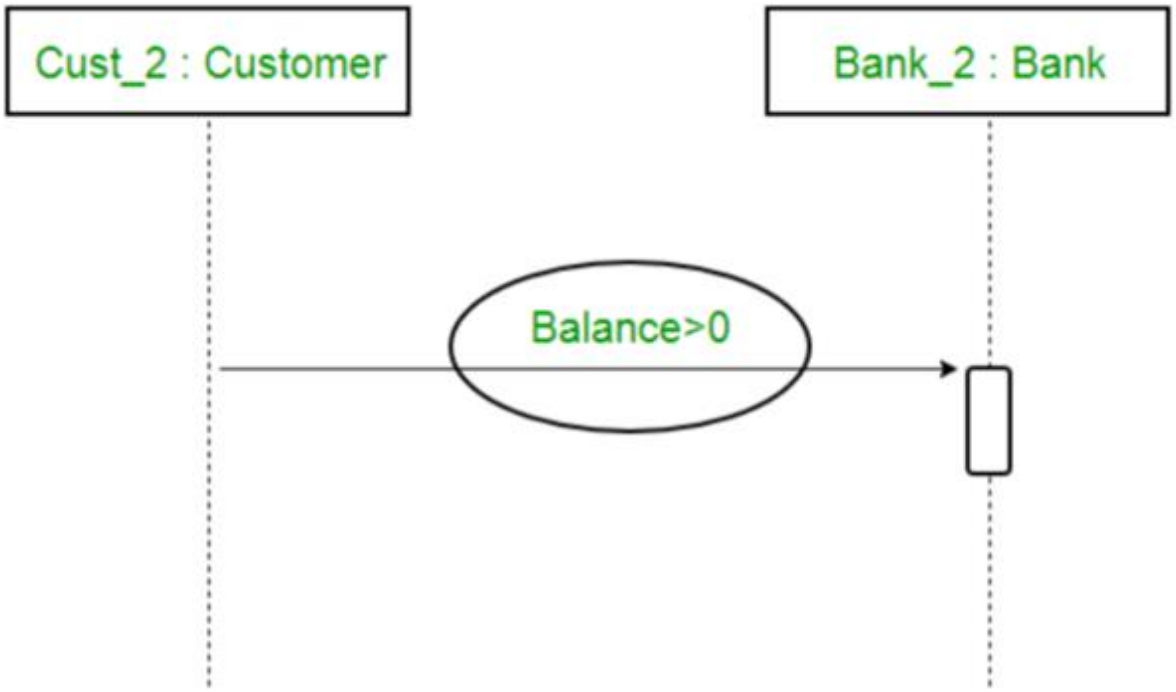
Lost Message:A Lost message is used to represent a scenario where the recipient is not known to the system. It is represented using an arrow directed towards an endpoint from a lifeline. For example: Consider a scenario where a warning is generated.

The warning might be generated for the user or other software/object that the lifeline is interacting with. Since the destination is not known beforehand, we use the Lost Message symbol.

Guards : To model conditions, we use guards in UML. They are used when we need to restrict the flow of messages on the pretext of a condition being met. Guards play an important role in letting software developers know the constraints attached to a system or a particular process.

For example: In order to be able to withdraw cash, having a balance greater than zero is a condition that must be met as shown below.

Figure: sequence diagram using a guard



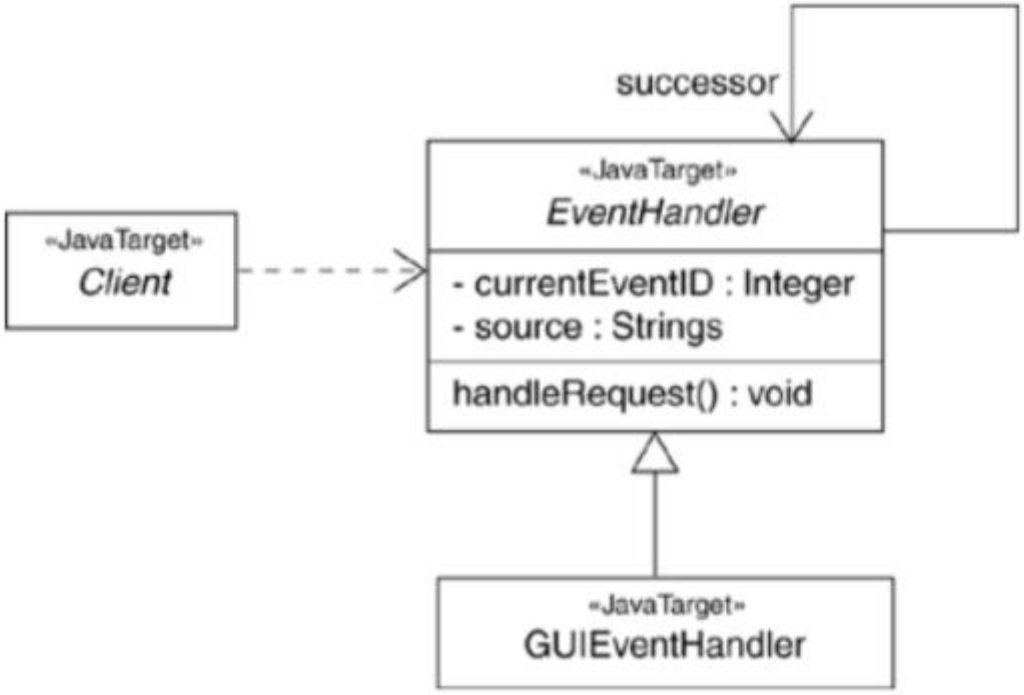
Forward engineering is the process of transforming a model into code through a mapping to an implementation language. It results in a loss of information, because models written in the UML are semantically richer than any current object-oriented programming language.

To forward engineer a class diagram,

·Identify the rules for mapping to your implementation language or languages of choice.

·Depending on the semantics of the languages you choose, you may want to constrain your use of certain UML features.

·Use tagged values to guide implementation choices in your target language.
·Use tools to generate code.



All of these classes specify a mapping to Java, as noted in their stereotype. Forward engineering the classes in this diagram to Java is straightforward, using a tool.
Forward engineering the class EventHandler yields the following code.

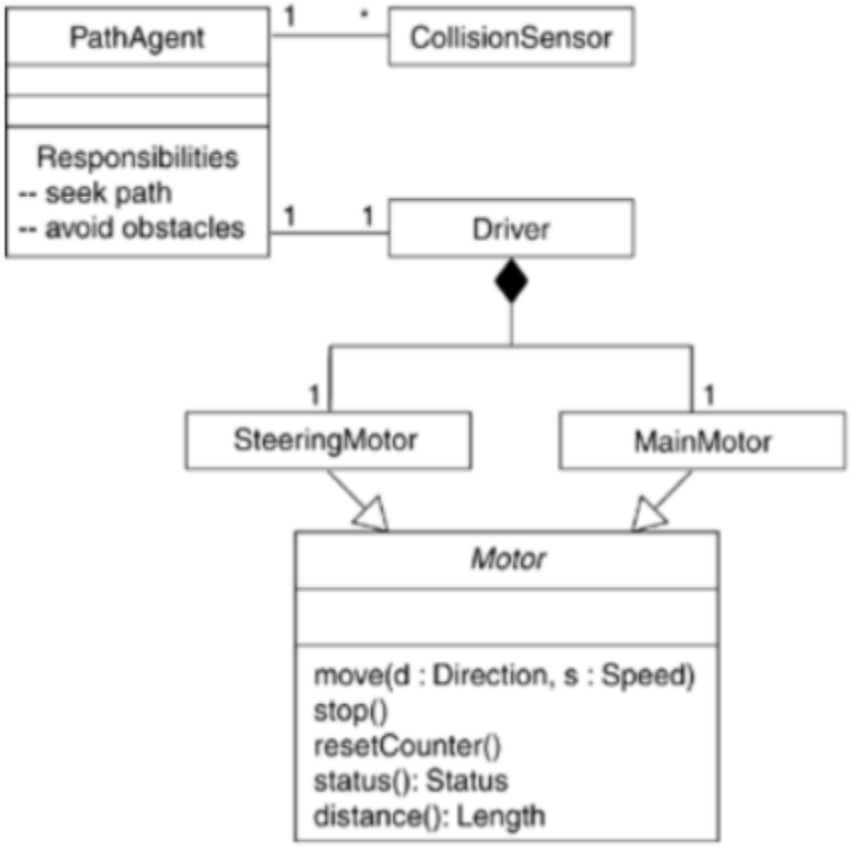
```
public abstract class EventHandler {


EventHandler successor;

private Integer currentEventID;

private String source;

EventHandler() {}

public void handleRequest() {}

}
```

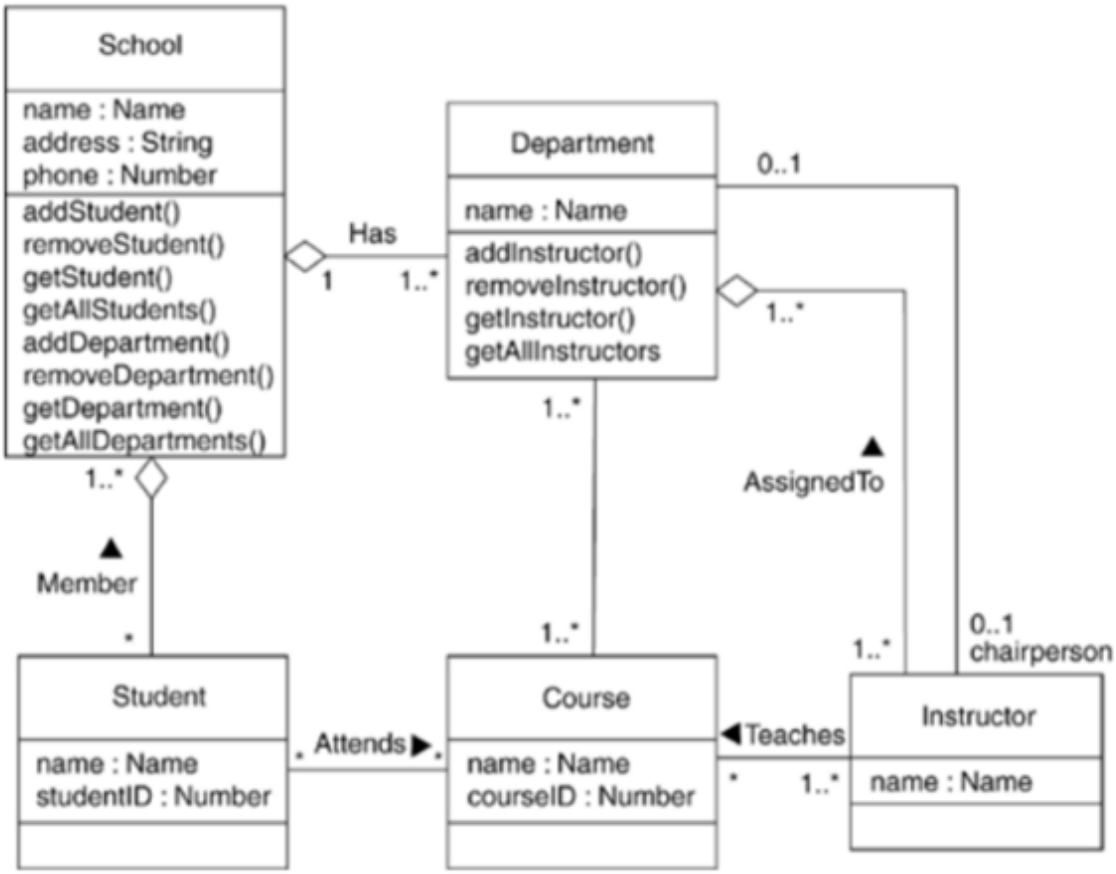
Reverse engineering is the process of transforming code into a model through a mapping from a specific implementation language. Reverse engineering results in a flood of information, some of which is at a lower level of detail than you'll need to build useful models. At the same time, reverse engineering is incomplete. There is a loss of information when forward engineering models into code, and so you can't completely recreate a model from code unless your tools encode information in the source comments that go beyond the semantics of the implementation language.

To reverse engineer a class diagram,

- Identify the rules for mapping from your implementation language or languages of choice.
- Using a tool, point to the code you'd like to reverse engineer. Use your tool to generate a new model or modify an existing one that was previously forward engineered.
- Using your tool, create a class diagram by querying the model.
- Manually add design information to the model to express the intent of the design that is missing or hidden in the code.

Some examples include:


PathAgent
Responsibilities
-- seek path
-- avoid obstacles
1
*
CollisionSensor
1
1
Driver
1
1
SteeringMotor
MainMotor
Motor
move(d : Direction, s : Speed)
stop()
resetCounter()
status(): Status
distance(): Length



To model complex views,

·There is no meaningful way to present this information at a higher level of abstraction, perhaps eliding some parts of the diagram and retaining the detail in other parts.

·If you've hidden as much detail as you can and your diagram is still complex, consider grouping some of the elements in packages or in higher-level collaborations, then render only those packages or collaborations in your diagram.

·If your diagram is still complex, use notes and color as visual cues to draw the reader's attention to the points you want to make.

·If your diagram is still complex, print it in its entirety and hang it on a convenient large wall. You lose the interactivity that an online version of the diagram brings, but you can step back from the diagram and study it for common patterns.

A relationship is a connection between things.Graphically, a relationship is rendered as a path, with different kinds of lines used to distinguish the kinds of relationship.

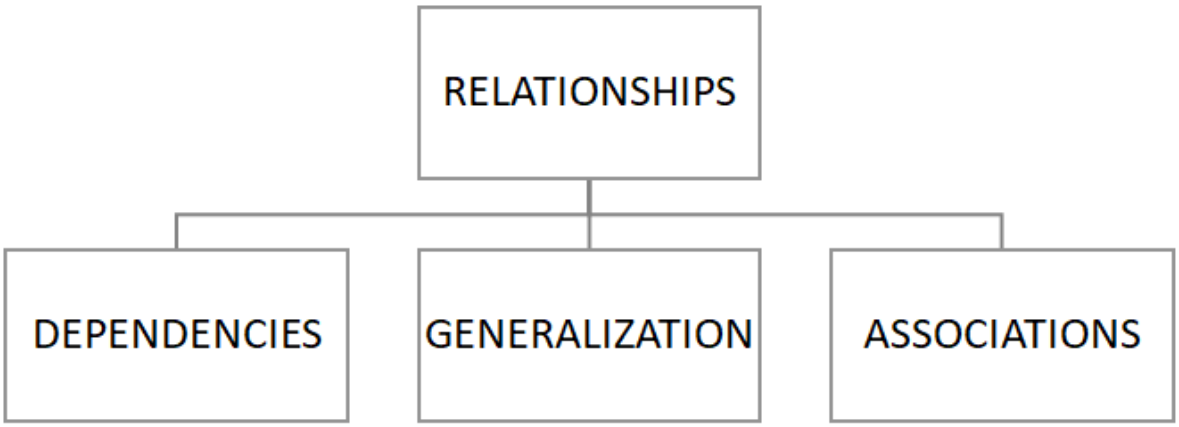
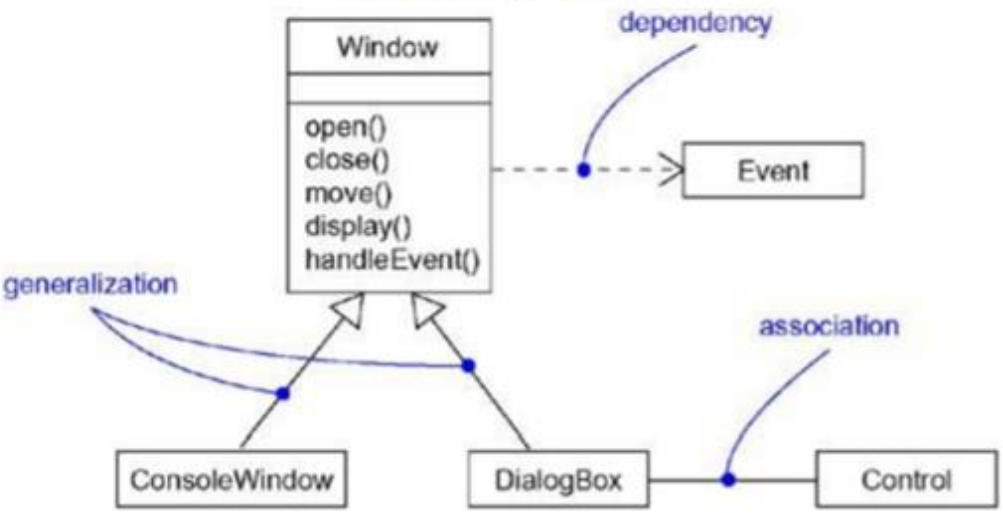


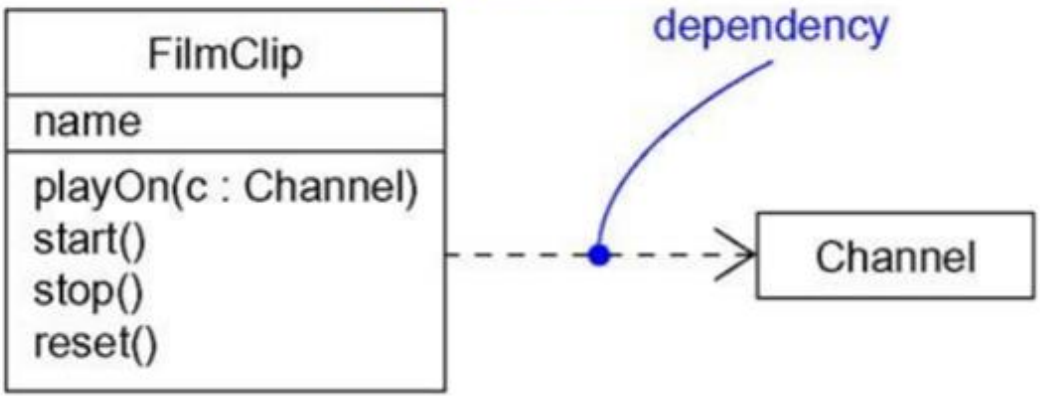
Figure 5-1 Relationships



1.Dependency

A dependency is a using relationship that states that a change in specification of one thing may affect another thing that uses it (for example, class Window), but not necessarily the reverse. Graphically, a dependency is rendered as a dashed directed line, directed to the thing being depended on.

Figure 5-2 Dependencies



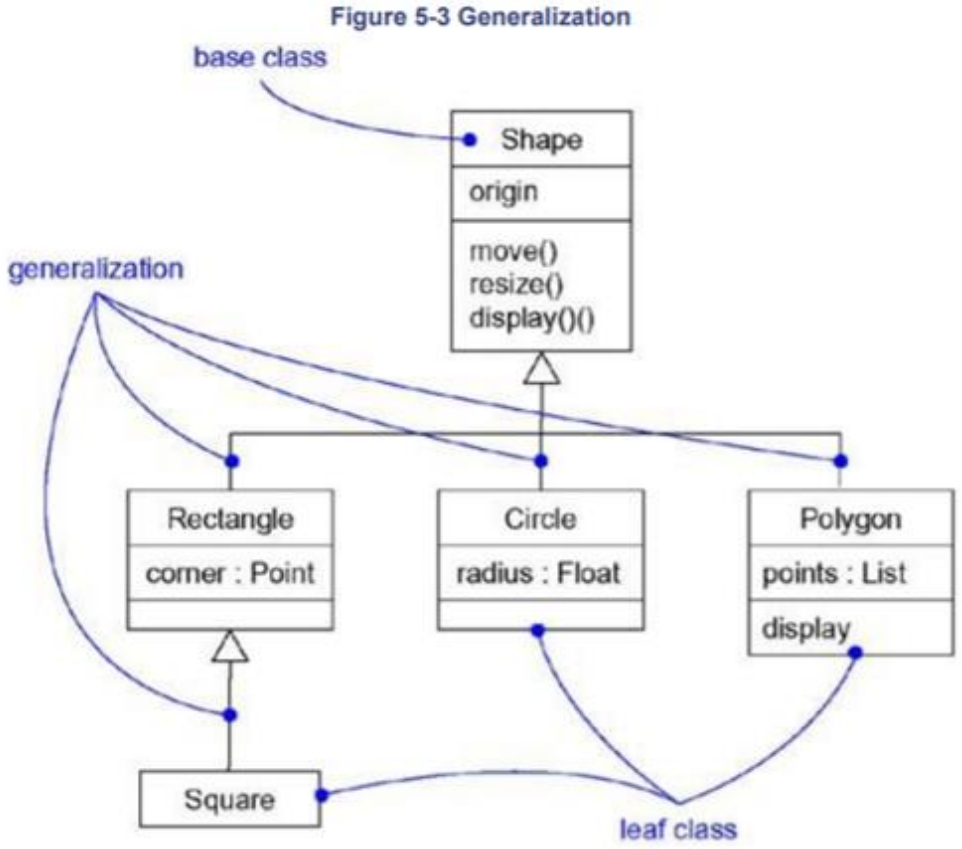
2.Generalization

A generalization is a relationship between a general thing (called the super class or parent)and a more specific kind of that thing (called the subclass or child).

Generalization is sometimes called an "is-a-kind-of" relationship: one thing is-a-kind-of a more general thing .

A child inherits the properties of its parents, especially their attributes and operations. An operation of a child that has the same signature as an operation in a parent overrides the operation of the parent, this is known as polymorphism.

Graphically, generalization is rendered as a solid directed line with a large open arrowhead, pointing to the parent, as shown in Figure.

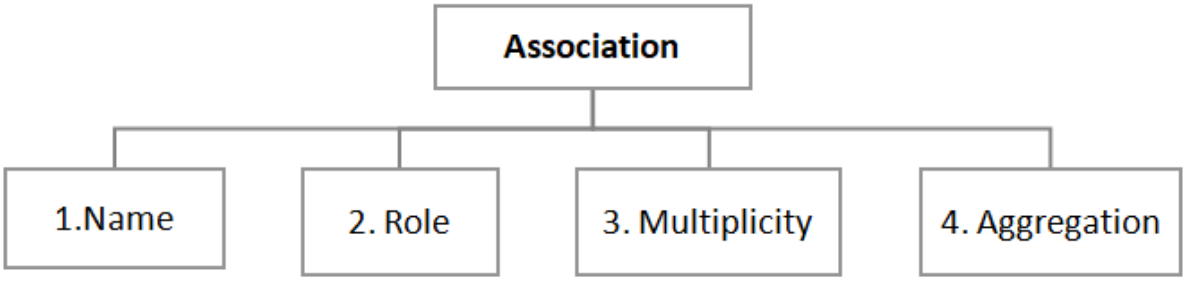


3.Association

An association is a structural relationship that specifies that objects of one thing are connected to objects of another. Given an association connecting two classes, you can navigate from an object of one class to an object of the other class, and vice versa. This means that, given an object of the class, you can link to other objects of the same class.

An association that connects exactly two classes is called a binary association. Although it's not as common, you can have associations that connect more than two classes; these are called n-ary associations. Graphically, an association is rendered as a solid line connecting the same or different classes.

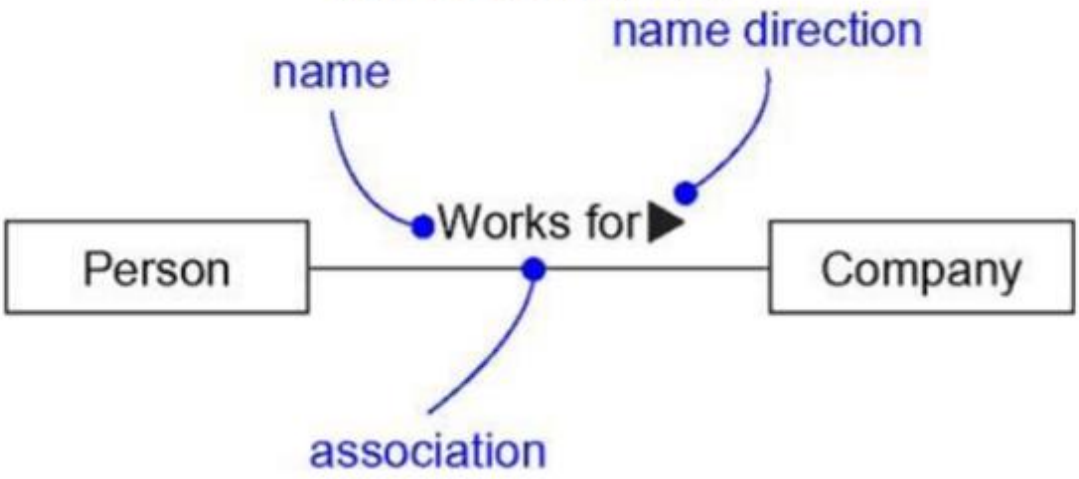
Beyond this basic form, there are four adornments that apply to associations i.e. Name, Role, Multiplicity and Aggregation.



1.Name

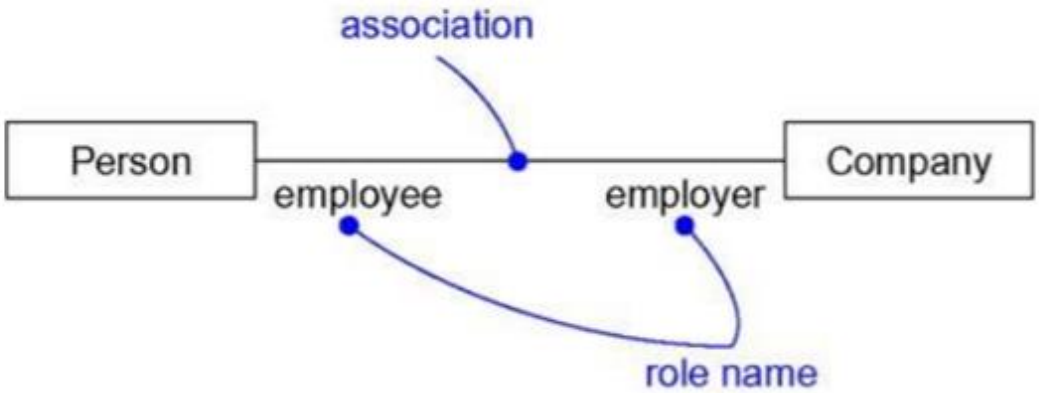
An association can have a name, and you use that name to describe the nature of the relationship. So that there is no ambiguity about its meaning, you can give a direction to the name by providing a direction triangle that points in the direction you intend to read the name.

Figure 5-4 Association Names



2.Role

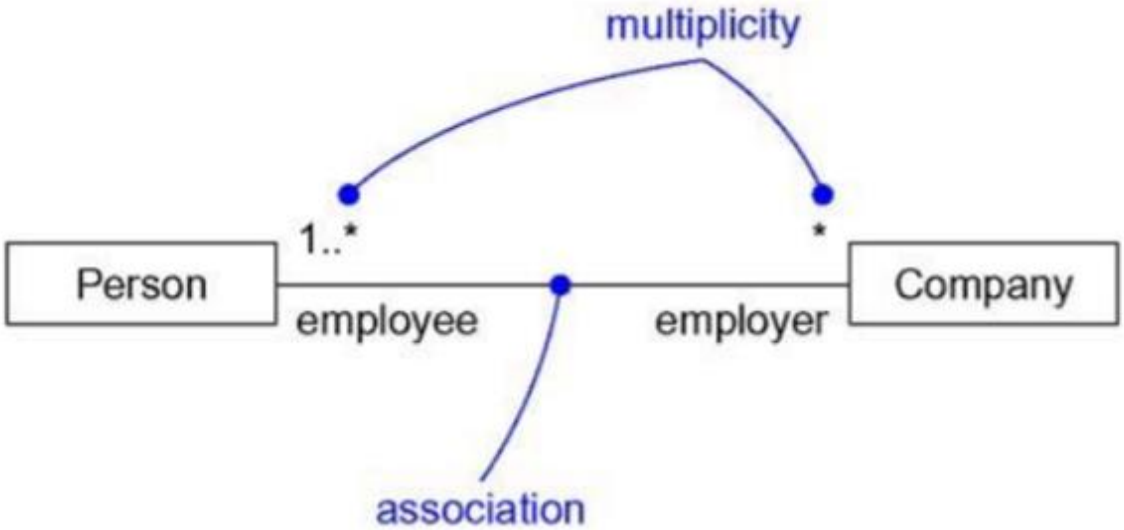
When a class participates in an association, it has a specific role that it plays in that relationship; a role is just the face the class at the near end of the association presents to the class at the other end of the association. You can explicitly name the role a class plays in an association. In Figure , a Person playing the role of employee is associated with a Company playing the role of employer.



3.Multiplicity

In many modeling situations, it's important for you to state how many objects may be connected across an instance of an association. This "how many" is called the multiplicity of an association's role, and is written as an expression that evaluates to a range of values or an explicit value as in Figure. When you state a multiplicity at one end of an association, you are specifying that, for each object of the class at the opposite end, there must be that many objects at the near end.

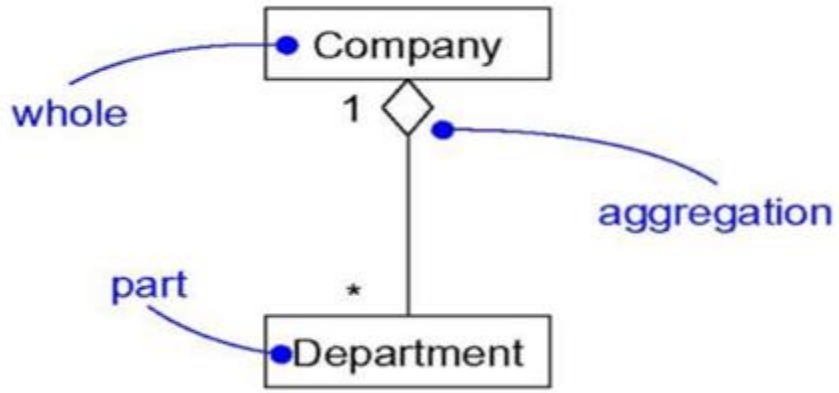
You can show a multiplicity of
exactly one (1),
zero or one (0..1),
many (0..), or*
one or more (1..).*
You can even state an exact number (for example, 3).



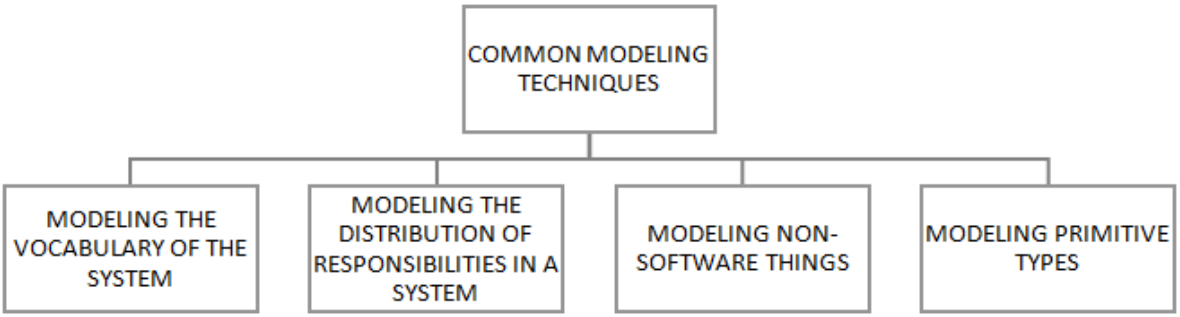
4.Aggregation

A plain association between two classes represents a structural relationship between peers, meaning that both classes are conceptually at the same level, no one more important than the other. Sometimes, you will want to model a "whole/part" relationship, in which one class represents a larger thing (the "whole"), which consists of smaller things (the "parts"). This kind of relationship is called aggregation, which represents a "has-a" relationship, meaning that an object of the whole has objects of the part.

Figure 5-7 Aggregation



The open diamond distinguishes the "whole" from the "part," no more, no less.



1.Modeling the Vocabulary of a System

You'll use classes most commonly to model abstractions that are drawn from the problem you are trying to solve or from the technology you are using to implement a solution to that problem.

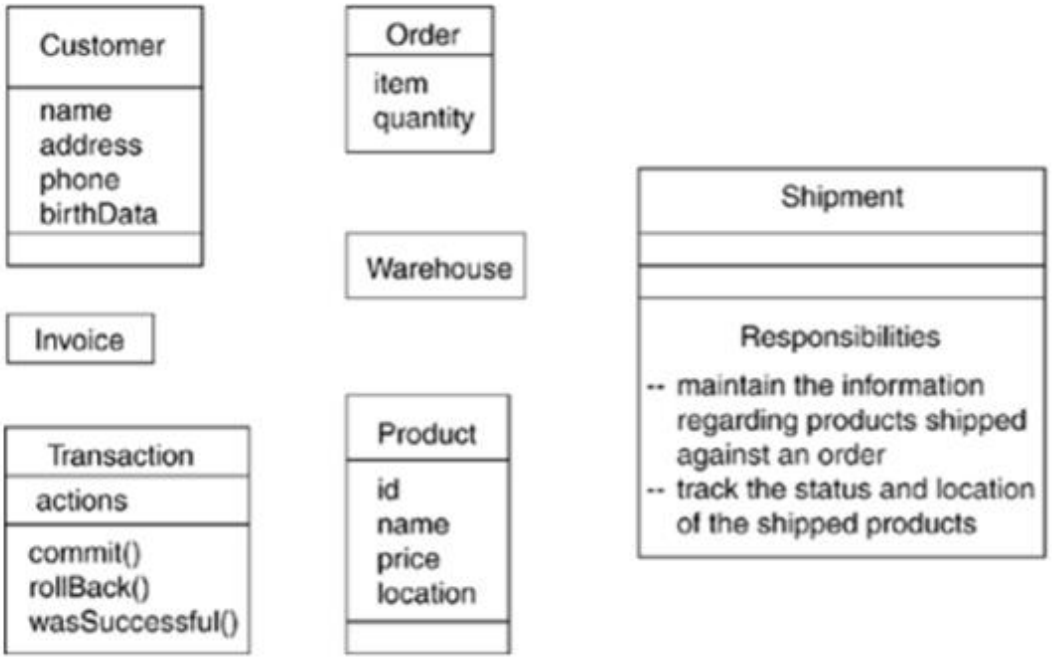
Each of these abstractions is a part of the vocabulary of your system, meaning that, together, they represent the things that are important to users and to implementers.

To model the vocabulary of a system,
-Identify those things that users or implementers use to describe the problem or solution.
-Use CRC cards and use case-based analysis to help find these abstractions.

-For each abstraction, identify a set of responsibilities.
-Make sure that each class is crisply defined and that there is a good balance of responsibilities among all your classes.

Provide the attributes and operations that are needed to carry out these responsibilities for each class.

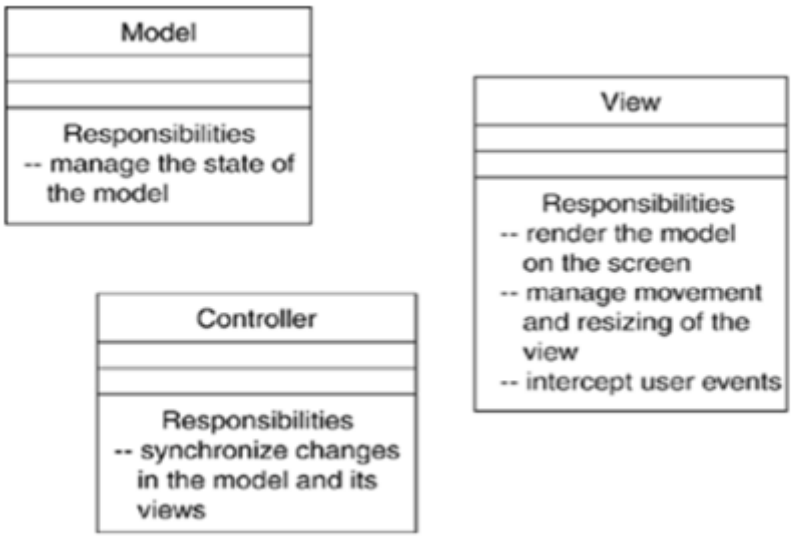
A set of classes drawn from a retail system, including Customer, Order, and Product. This figure includes a few other related abstractions drawn from the vocabulary of the problem, such as Shipment, Invoice,and Warehouse.



2. Modeling the Distribution of Responsibilities in a System

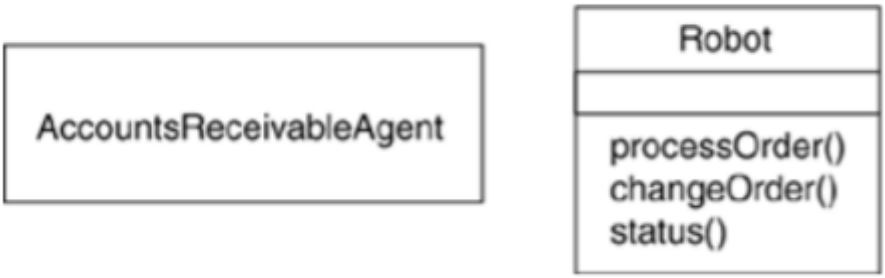
Once you start modeling more than just a handful of classes, you will want to be sure that your abstractions provide a balanced set of responsibilities.

To model the distribution of responsibilities in a system,
-Identify a set of classes that work together closely to carry out some behavior.
-Identify a set of responsibilities for each of these classes.
-Look at this set of classes as a whole, split classes that have too many responsibilities into smaller abstractions, collapse tiny classes that have trivial responsibilities into larger ones, and reallocate responsibilities so that each abstraction reasonably stands on its own.
-Consider the ways in which those classes collaborate with one another, and redistribute their responsibilities accordingly so that no class within a collaboration does too much or too little.



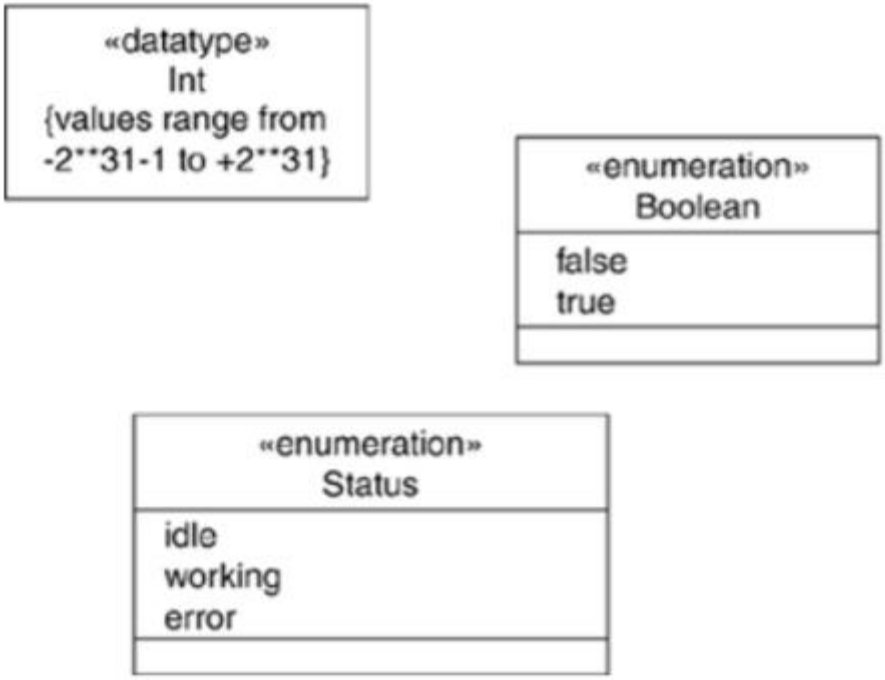
3. Modeling Non-software Things

To model non-software things,
-Model the thing you are abstracting as a class.
-If you want to distinguish these things from the UML's defined building blocks, create a new building block by using stereotypes to specify these new semantics and to give a distinctive visual cue.
-If the thing you are modeling is some kind of hardware that itself contains software, consider modeling it as a kind of node as well, so that you can further expand on its structure.



4. Modeling Primitive Types

To model primitive types,
-Model the thing you are abstracting as a class or an enumeration, which is rendered using class notation with the appropriate stereotype.
-If you need to specify the range of values associated with this type, use constraints.


«datatype»
Int
{values range from
-2**31-1 to +2**31}
«enumeration»
Boolean
false
true
«enumeration»
Status
idle
working
error